PROJECT

Credit Card Fraud Detection UsingMachine Learning

This is Submitted By:

Rizzu Khan -- 12417714

Roll No. -- 23

B.Tech CSE School of Computing & Artificial Intelligence

Lovely Professional University

Submitted To -- Tanima Thakur



Declaration

I hereby declare that the project report entitled “Credit Card Fraud Detection Using Machine Learning”, submitted to the School of Computing & Artificial Intelligence, Lovely Professional University, is a record of original work done by me, Rizzu Khan (Registration No. 12417714), under the guidance of **Dr. Tanima Thakur**. The experiments and results reported were obtained by running the project code on the dataset described in this report, and no result has been copied or fabricated.

All sources of information and prior work that have been used are acknowledged in the References. This report has not been submitted, in part or in full, for any other degree or diploma.

Rizzu Khan

Registration No.: 12417714

Abstract

Credit card fraud causes direct financial loss, but fraudulent transactions are extremely rare and hard to separate from genuine ones. This project uses supervised and unsupervised machine learning to detect fraud in the Kaggle “Credit Card Fraud Detection” dataset, which contains 284,807 transactions made by European cardholders, of which only 473 are fraudulent after duplicate removal (0.167%). The target variable, Class, is 0 for a genuine and 1 for a fraudulent transaction.

The data were cleaned (1,081 duplicate rows removed), explored with graphs, and reduced to a working sample of all 473 fraud cases and 5,000 randomly chosen genuine transactions (5,473 rows), because distance-based and kernel-based models are slow on the full data. The sample was split 80:20 into 4,378 training and 1,095 test records with stratification, and the features were standardised using StandardScaler. Five classifiers from the syllabus were trained and compared: Logistic Regression, k-Nearest Neighbours, Gaussian Naive Bayes,

Decision Tree and Support Vector Machine. They were evaluated with accuracy, precision, recall, F1-score, ROC AUC, logarithmic loss and a confusion matrix.

The SVM achieved the highest accuracy (0.9817), F1-score (0.8837) and ROC AUC (0.9847); Logistic Regression was almost identical (F1 0.8814, AUC 0.9826) and had the lowest log loss (0.0671). KNN and Decision Tree followed, and Gaussian Naive Bayes was the weakest (accuracy 0.9644, F1 0.8000, log loss 0.8224). The SVM confusion matrix shows 999 of 1,000 genuine and 76 of 95 fraud transactions classified correctly. The project also applies simple, polynomial and multiple linear regression to predict the transaction Amount (best $R^2 = 0.8980$ for multiple linear regression), and K-Means and hierarchical clustering to the V1–V28 features. K-Means produced one cluster that contained only fraud cases (191 of 473).

The study is limited to one dataset, one train-test split, a rebalanced sample and untuned models. Possible improvements are given as future work.

Keywords: credit card fraud, classification, imbalanced data, Logistic Regression, SVM, KNN, Decision Tree, Naive Bayes, regression, K-Means, hierarchical clustering.

1. Literature Review

1.1 Purpose of the Review

This chapter surveys the published work and documentation on which the project rests: credit card fraud detection and imbalanced data, the source of the dataset, the five classifiers, the regression and clustering methods, and the evaluation measures. It is limited to sources that are directly relevant to what was implemented. Every source is listed in the References chapter, and a summary table at the end of the chapter shows which part of the project each source supports.

1.2 Credit Card Fraud and Imbalanced Data

Bolton and Hand [9] reviewed statistical and data-mining approaches to fraud detection and pointed out that fraud problems are difficult because fraud cases are rare, because the cost of a missed fraud differs from the cost of a false alarm, and because fraudsters change their behaviour over time. A direct consequence

of rarity is class imbalance: a classifier that labels every transaction as genuine is almost always “correct”. Accuracy alone is therefore a poor measure for this problem, and precision, recall, F1-score and ROC AUC are used alongside it [7].

Two common ways of dealing with imbalance are to under-sample the majority class and to over-sample the minority class; the SMOTE method of Chawla et al. [16] creates synthetic minority examples instead of simply copying them. Dal Pozzolo et al. [10], the group whose data are used here, studied under-sampling for unbalanced classification and showed that it changes the class proportions seen by the model and hence the predicted probabilities. This project uses a simple form of under-sampling (all fraud cases and a random subset of genuine cases), and Section 11.7 discusses what this means for the interpretation of the results.

1.3 Source of the Dataset

The dataset was collected by the Machine Learning Group of Université Libre de Bruxelles together with Worldline and is published on Kaggle [11]. It contains transactions made by European cardholders over two days in September 2013. For confidentiality, the original input variables were transformed by principal component analysis (PCA) and are published as V1 to V28; only Time and Amount are given in their original form [11].

1.4 Literature on the Five Classifiers

Logistic Regression. Hosmer, Lemeshow and Sturdivant [5] give the standard treatment of the model, which passes a weighted sum of the features through the sigmoid function to produce a probability. It is fast, stable and easy to interpret, which is why it is often the first baseline in a classification study.

k-Nearest Neighbours. Cover and Hart [3] analysed the nearest-neighbour rule and showed that, for a large training set, its error rate is at most twice the Bayes error rate. Because it relies on distances, it is sensitive to feature scale, and standardisation is a standard step before using it [14].

Naive Bayes. The classifier applies Bayes’ theorem under the assumption that features are independent given the class. Domingos and Pazzani [6] studied why it often classifies well even when this assumption is false, although its probability estimates can then be poor.

Decision Tree. Breiman, Friedman, Olshen and Stone [4] introduced Classification and Regression Trees (CART), which split the data recursively by simple threshold rules chosen to make the groups as pure as possible. They also discuss the tendency of an unrestricted tree to over-fit, which motivates limiting its depth.

Support Vector Machine. Cortes and Vapnik [2] introduced the support-vector network, which finds a boundary with a large margin, tolerates some misclassified points through a soft margin, and uses kernel functions for non-linear boundaries.

1.5 Literature on Regression and Clustering

Linear, multiple and polynomial regression, fitted by ordinary least squares, and the error measures MAE, MSE, RMSE and R^2 are treated in James, Witten, Hastie and Tibshirani [15] and Hastie, Tibshirani and Friedman [14]. K-Means clustering goes back to MacQueen [12]; because its result depends on the starting centroids, Arthur and Vassilvitskii [13] proposed k-means++, which spreads the initial centroids and usually gives better and more stable clusters. Agglomerative hierarchical clustering and the single, complete, average and centroid linkages are described in [14] and [15]. Tukey [8] established exploratory data analysis, in which graphs are used to understand the data before modelling; the EDA chapter of this report follows that idea.

1.6 Summary of Sources and Research Gap

Ref.	Source type	Used in this project for
[1]	Journal paper	scikit-learn tools: scaling, splitting, models, metrics
[2]	Journal paper	Support Vector Machine theory
[3]	Journal paper	k-Nearest Neighbours theory
[4]	Book	Decision Tree (CART) theory
[5]	Book	Logistic Regression theory
[6]	Journal paper	Why Naive Bayes works despite its assumption
[7]	Journal paper	Precision, recall, F1 and their meaning

Ref.	Source type	Used in this project for
[8]	Book	Exploratory data analysis
[9]	Journal paper	Nature of the fraud-detection problem
[10], [11]	Conference paper, dataset page	Origin and description of the dataset
[12], [13]	Conference papers	K-Means and k-means++ initialisation
[14], [15]	Textbooks	Regression, classification and clustering background
[16]	Journal paper	Handling class imbalance (SMOTE)

The literature shows that the algorithms used here are mature and suitable for tabular data, and that fraud detection is an imbalanced problem. What it does not provide is a small, syllabus-level comparison that puts classification, regression and clustering on one fraud dataset and explains every step. This project therefore offers a reproducible comparison of five classifiers, three regression models and two clustering methods on one dataset. It does not claim to improve on any published method.

2. Introduction

2.1 Background

Credit cards are used for billions of payments, and fraudulent use of a card or its details causes losses to customers, banks and merchants. Banks therefore run automated systems that score every transaction and flag suspicious ones for review. Such systems must work on data in which fraud is very rare, so a method that looks accurate can still miss most of the fraud.

Machine learning offers a systematic way to build these systems. A supervised classifier is shown many transactions whose label (genuine or fraud) is known, learns the pattern that separates them, and is then tested on transactions it has not seen. Unsupervised methods, which do not use the label, can be used to group similar transactions and to see whether fraud forms a group of its own.

2.2 Motivation

The motivation is two-fold. First, the project is a practical application of the topics of the course: data preprocessing, regression, classification and clustering, together with the standard evaluation measures. Second, fraud data show clearly why a single accuracy number is not enough: the classes are extremely unequal, the cost of a missed fraud is high, and the "best" model depends on the measure chosen. These lessons are visible in the results of this project (Chapter 11).

2.3 Overview of the Project

The project uses the Credit Card Fraud Detection dataset from Kaggle (284,807 transactions, 31 columns). The data were cleaned and reduced to a sample containing all fraud cases and 5,000 genuine transactions. Five classifiers were trained and compared: Logistic Regression, KNN, Gaussian Naive Bayes, Decision Tree and SVM. They were evaluated with accuracy, precision, recall, F1-score, ROC AUC, logarithmic loss and a confusion matrix. In addition, simple, polynomial and multiple linear regression were used to predict the transaction Amount, and K-Means and hierarchical clustering were applied to the V1–V28 features.

2.4 Scope and Syllabus Boundary

The project is restricted to topics of Units I to IV of the syllabus, as shown below. Association rules and market basket analysis (Unit IV) are not used, because they need item-level basket data, which this dataset does not contain. Techniques outside the syllabus, such as ensemble methods, neural networks and automated hyper-parameter search, are not part of the implemented work and appear only in the Future Scope chapter.

Unit	Syllabus topic	Where it appears in this project
I	Data preprocessing, supervised and unsupervised learning	Chapter 6; whole project
II	Logistic Regression	Classifier 1 (Chapter 9)
II	Simple, multiple and polynomial regression; correlation	Section 11.8; Figure 2

Unit	Syllabus topic	Where it appears in this project
II	MAE, MSE, RMSE, R^2	Section 11.8
III	KNN, Naive Bayes, Decision Tree, SVM	Classifiers 2 to 5
III	Accuracy, log loss, AUC, precision, recall, F1, confusion matrix	Sections 11.1 to 11.6
IV	K-Means, random initialisation, choosing K	Section 11.9
IV	Hierarchical clustering and linkages	Section 11.9
IV	Association rules, market basket analysis	Not used (see above)

2.5 Organisation of the Report

Chapter 1 reviews the literature and Chapter 2 introduces the project. Chapters 3 and 4 state the problem and objectives. Chapter 5 describes the dataset and Chapter 6 the cleaning and preprocessing. Chapter 7 presents the exploratory data analysis. Chapter 8 explains the methodology and evaluation measures, Chapter 9 the algorithms, and Chapter 10 the implementation and the code. Chapter 11 presents the results, Chapter 12 the conclusions and Chapter 13 the future scope. References and appendices follow.

3. Problem Statement

3.1 Formal Statement

Given the attributes of a credit card transaction (the time, the amount and 28 anonymised numerical components V1 to V28), predict whether the transaction is genuine (Class = 0) or fraudulent (Class = 1). This is a binary supervised classification problem. Let x be the feature vector of a transaction after cleaning and scaling, and y its class. The aim is to learn a function f such that f(x) is as close as possible to y for transactions that were not used in training.

```
f : x \rightarrow y, \qquad y \in \{0, 1\}
```

Two further tasks are studied on the same data: predicting the numerical Amount of a transaction (regression), and grouping transactions without using the label (clustering).

3.2 Why the Problem Is Non-Trivial

- **Extreme imbalance.** Only 473 of 283,726 cleaned transactions (0.167%) are fraud, so a model that always predicts "genuine" would be right more than 99.8% of the time and still useless.
- **Costs are unequal.** A missed fraud costs the amount stolen, whereas a false alarm costs only a review or a blocked card, so recall and precision matter more than accuracy.
- **Anonymised features.** V1 to V28 are PCA components, so they cannot be interpreted as business quantities such as merchant or country.
- **Different scales.** Time and Amount have very different ranges from the V features, which affects distance-based and margin-based models unless the data are standardised.

3.3 Assumptions

- The cleaned dataset is treated as a representative sample of the transactions it describes; the results are not claimed to generalise beyond it.
- The Class column is taken as the ground-truth label.
- All five classifiers are compared on the same sample, the same split and the same measures.

3.4 Out of Scope

The project does not attempt real-time deployment, does not use data outside the supplied dataset, and does not make any claim about individual cardholders. It also does not tune the models or handle imbalance beyond the sampling step described in Chapter 6.

4. Objectives

4.1 Main Objective

To build and compare five supervised classification models that predict whether a credit card transaction is fraudulent, to apply regression and clustering to the same data, and to identify, using standard evaluation measures, which classifier performs best.

4.2 Specific Objectives

1. To understand the dataset, its variables and its target Class, and to verify their meaning from the dataset documentation.
2. To clean the data by removing duplicate records and checking for missing values.
3. To explore the data with graphs: the class distribution, the features most correlated with fraud, and the amount distribution.
4. To prepare the data for modelling by sampling, splitting and standardising with StandardScaler.
5. To implement and train Logistic Regression, KNN, Gaussian Naive Bayes, Decision Tree and SVM.
6. To evaluate each classifier with accuracy, precision, recall, F1-score, ROC AUC and log loss, and to examine the confusion matrix.
7. To apply simple, polynomial and multiple linear regression to predict Amount and to evaluate them with MAE, MSE, RMSE and R^2.
8. To apply K-Means (with the elbow method) and hierarchical clustering (four linkages) and to compare the clusters with the actual classes.
9. To compare the results in tables and graphs and to state honest conclusions and limitations.

4.3 Completed Work and Future Work

Category	Items
Completed (Chapters 5 to 11)	Dataset understanding; cleaning; sampling; EDA; scaling; train-test split; five classifiers with six measures; confusion matrix; three regression models; K-Means with the elbow

Category	Items
	method and initialisation comparison; hierarchical clustering with four linkages.
Not completed (Chapter 13 only)	Cross-validation; hyper-parameter tuning; other imbalance methods such as SMOTE or class weights; ensemble or neural models; testing on the full, unsampled data; association rules.

5. Dataset Description

5.1 Source and Coverage

The dataset is the "Credit Card Fraud Detection" dataset published on Kaggle by the Machine Learning Group of Université Libre de Bruxelles (ULB): https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud [11]. It contains transactions made with credit cards by European cardholders in September 2013. The Time column runs from 0 to 172,792 seconds, i.e. about 48 hours, so the data cover two days. The file used in this project is creditcard.csv. Each row is one transaction.

5.2 Target Variable: Class

Class is a binary variable: 0 for a genuine transaction and 1 for a fraudulent one.

Class	Meaning	Count after cleaning	Share of cleaned data	In the 5,473-row sample	In the test set
0	Genuine transaction	283,253	99.83%	5,000	1,000
1	Fraudulent transaction	473	0.17%	473	95
Total		283,726	100%	5,473	1,095

The test-set counts are the row totals of the SVM confusion matrix (Figure 10). The dataset documentation reports 492 fraud cases before cleaning; 473 remain after duplicate rows are removed.

5.3 Features

The dataset has 31 columns: Time, V1 to V28, Amount and Class.

Feature	Meaning	Type
Time	Seconds elapsed between this transaction and the first transaction in the dataset	Numeric (original)
V1 – V28	Principal components obtained by PCA from the original, confidential transaction attributes	Numeric (anonymised)
Amount	Transaction amount	Numeric (original)
Class	0 = genuine, 1 = fraud (target)	Binary

The summary statistics printed by the program for the first columns (before cleaning) are shown below.

Column	Mean	Std. deviation	Minimum	Maximum
Time	94,813.86	47,488.15	0.00	172,792.00
V1	≈ 0	1.9587	−56.4075	2.4549
V2	≈ 0	1.6513	−72.7153	22.0577
V3	≈ 0	1.5163	−48.3256	9.3826
V4	≈ 0	1.4159	−5.6832	16.8753
V5	≈ 0	1.3802	−113.7433	34.8017
V6	≈ 0	1.3323	−26.1605	73.3016
V7	≈ 0	1.2371	−43.5572	120.5895
V8	≈ 0	1.1944	−73.2167	20.0072
V9	≈ 0	1.0986	−13.4341	15.5950

The V columns have a mean of almost exactly zero, which is expected for PCA components. Their standard deviations are close to 1 to 2, whereas Time has a standard deviation of about 47,488, so the features are on very different scales.

5.4 Basic Statistics

- Records before cleaning: 284,807; columns: 31.
- Records after cleaning: 283,726 (1,081 duplicate rows removed, derived as 284,807 − 283,726); columns: 31.
- Missing values: 0.
- Fraud share: 473 / 283,726 = 0.167% in the cleaned data and 473 / 5,473 = 8.64% in the sample used for modelling.

5.5 Limitations of the Dataset

- **Anonymised features.** The V columns cannot be interpreted, so the report cannot say which real-world behaviour makes a transaction suspicious.
- **Short period.** The data cover only two days, so changes in fraud patterns over time are not studied.
- **One region and period.** The transactions are from European cardholders in 2013, and patterns may differ today or elsewhere.
- **Very few fraud cases.** With only 473 fraud cases (95 in the test set), each fraud case changes recall by about one percentage point.
- **PCA was computed by the data providers.** The details of the transformation are not available, so the V features cannot be re-created from raw data.

6. Data Cleaning and Preprocessing

Real data rarely arrive in a form that can be given directly to a machine-learning algorithm. This chapter explains each preprocessing step carried out in the program, why it is needed and what the program found.

6.1 Missing Values

A missing value is a cell in which nothing was recorded. Most scikit-learn models stop with an error if they receive one. The program removes rows with a missing value (dropna) and then counts what is left (isnull().sum()). The count printed was **0**, so no rows were lost and no imputation was needed.

6.2 Duplicate Removal

A duplicate record is a row identical to another row. Duplicates give an observation extra weight, and if one copy falls in the training set and its twin in

the test set, the model is partly tested on data it has already seen, which makes the score look better than it is. The program removed duplicates with drop_duplicates(). The number of rows fell from 284,807 to **283,726**, so **1,081** duplicate rows were removed. Of the 492 fraud cases reported in the dataset documentation, 473 remained, and 283,253 genuine transactions remained.

6.3 Why No Encoding Was Needed

One-hot encoding is used when a dataset has categorical (text) columns. All 30 input columns of this dataset (Time, Amount and V1 to V28) are numeric, so no encoding step was required. Class is already stored as 0 and 1 and was converted to an integer type.

6.4 Creating the Working Sample

The cleaned data contain 283,726 rows. KNN has to compute distances to all training rows for every prediction, and an SVM with probability estimates takes a long time on hundreds of thousands of rows, so the program works on a sample:

1. all 473 fraud transactions are kept;
2. 5,000 genuine transactions are drawn at random (random_state = 42);
3. the two parts are joined and shuffled.

The sample has **5,473** rows (5,000 genuine, 473 fraud). The fraud share rises from 0.17% in the cleaned data to 8.64% in the sample. This is a form of under-sampling of the majority class [10], and it affects how the results should be read (Section 11.7).

6.5 Feature Scaling with StandardScaler

Time has values up to 172,792 and Amount up to several thousand, whereas the V columns are mostly between −10 and +10. KNN and SVM depend on distances and margins, and Logistic Regression is fitted by an iterative numerical method, so features with large ranges would dominate. StandardScaler rescales each feature to mean 0 and standard deviation 1:

```
z = \frac{x - \mu}{\sigma}
```

where μ and σ are the mean and standard deviation of the feature. In the program the scaler is fitted on the **training data only** (fit_transform on X_train) and the same transformation is then applied to the test data (transform on

X_test), so no information from the test set leaks into training. The same scaled data are used for all five classifiers so that the comparison is fair, although Decision Trees and Naive Bayes do not need scaling.

6.6 Train-Test Split

A model that is evaluated on the data it was trained on only shows that it can memorise. The sample was therefore divided with train_test_split into a **training set of 4,378 records** and a **test set of 1,095 records** (test_size = 0.20, random_state = 42). The split is **stratified** on Class (stratify = y), so both parts keep the same fraud share. The test set contains 1,000 genuine and 95 fraud transactions (Chapter 11).

6.7 Preparation for Regression and Clustering

- **Regression.** The target is Amount and the inputs are Time and V1 to V28. The sample was split 80:20 with random_state = 42 (not stratified, because the target is a number). No scaling was applied for regression.
- **Clustering.** V1 to V28 of the whole sample were standardised with a separate StandardScaler. The Class label is not used to form the clusters; it is used only afterwards to check how well the clusters match the real classes.

6.8 Summary of Preprocessing

Step	Problem it solves	Result in this project
Missing-value check	Empty cells that models cannot process	0 missing values
Duplicate removal	Repeated rows and test leakage	1,081 rows removed (283,726 remain)
Sampling	Long run-time of KNN and SVM on 283,726 rows	5,473 rows (473 fraud + 5,000 genuine)
Train-test split	Need to test on unseen data	4,378 training and 1,095 test records, stratified

Step	Problem it solves	Result in this project
StandardScaler	Different feature ranges	Mean 0, standard deviation 1, fitted on training data only

7. Exploratory Data Analysis

Exploratory data analysis (EDA) means looking at the data with simple summaries and graphs before modelling, in the spirit of Tukey [8]. This chapter presents the three EDA figures produced by the program and states only what can be read from them. Values read from a graph are approximate.

7.1 Distribution of the Target Variable

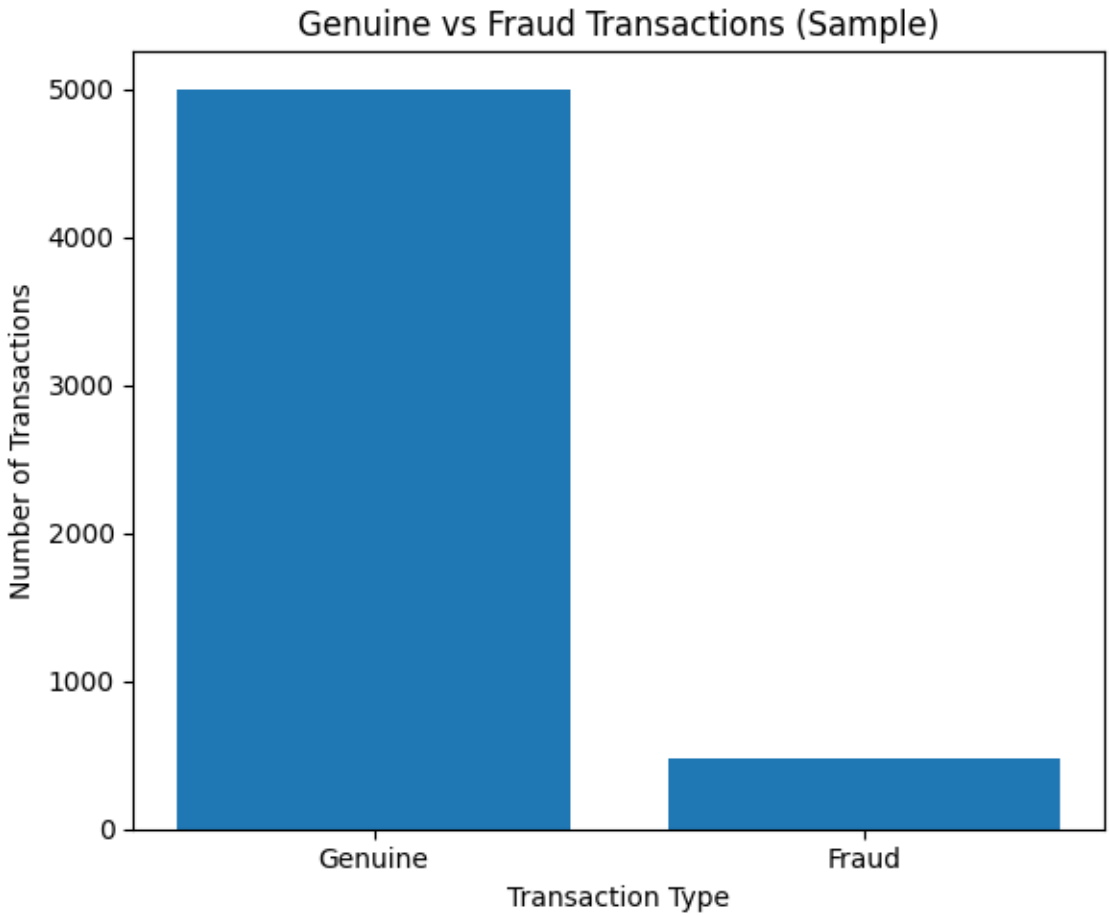


Figure 1: Genuine vs fraud transactions in the working sample

Figure 1 is a bar chart of the two classes in the 5,473-row sample. The genuine bar is at 5,000 and the fraud bar at 473, so genuine transactions outnumber fraud by about 10.6 to 1 (5,000 / 473). In the full cleaned data the ratio is far larger: 283,253 genuine to 473 fraud, about 599 to 1.

Why this matters. Because the classes are unequal, the share of the larger class is a reference point for accuracy. In the test set, 1,000 of 1,095 transactions are genuine, so a rule that predicts "genuine" for every transaction would already reach 91.3% accuracy (derived). All five models in Chapter 11 score above this level, but the margin is small, and accuracy alone is therefore not enough to judge the models.

7.2 Features Most Correlated with Fraud

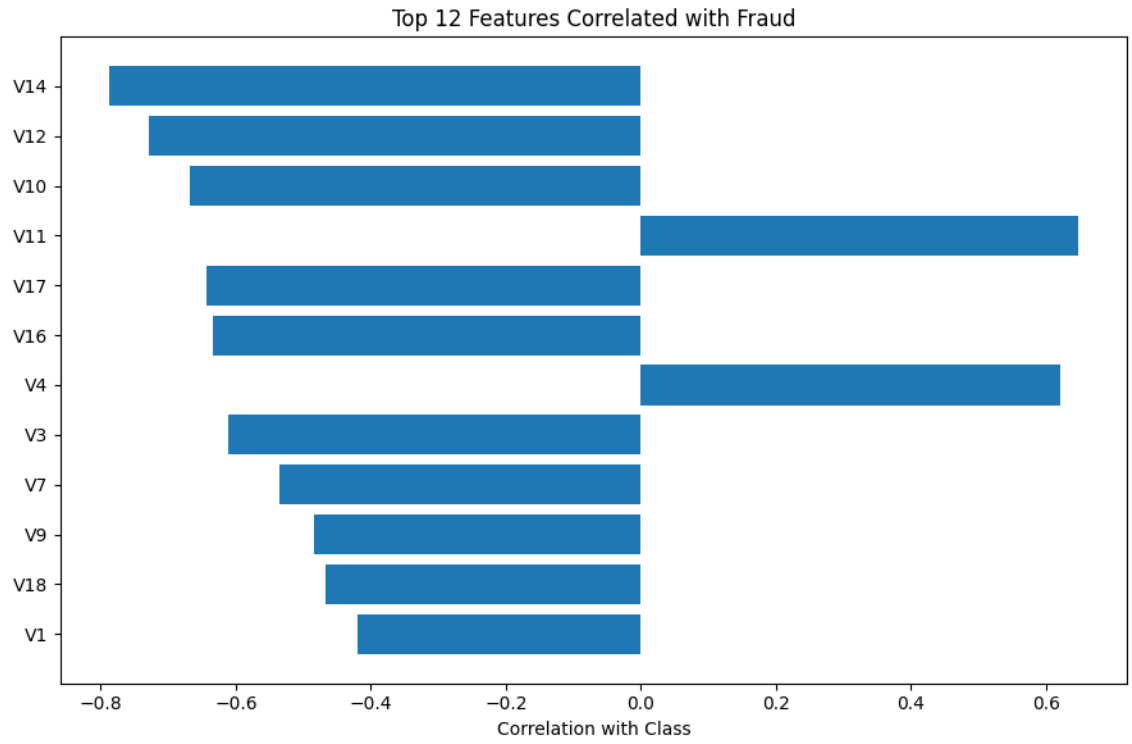


Figure 2: Top 12 features correlated with fraud (Class)

Figure 2 shows the 12 features with the largest absolute correlation with Class, computed on the sample. Approximate values read from the chart:

- **Negative correlation** (lower value, more likely fraud): V14 (about −0.79, the strongest), V12 (−0.73), V10 (−0.67), V17 (−0.64), V16 (−0.63), V3 (−0.61), V7 (−0.53), V9 (−0.48), V18 (−0.47) and V1 (−0.42).

- **Positive correlation** (higher value, more likely fraud): V11 (about +0.65) and V4 (+0.62).
- Time and Amount do not appear among the top 12, so the anonymised V components carry most of the linear signal.

A correlation shows a linear association, not a cause. Also, because the sample contains 8.64% fraud instead of 0.17%, the correlations are larger than they would be on the full data.

7.3 Distribution of Transaction Amount

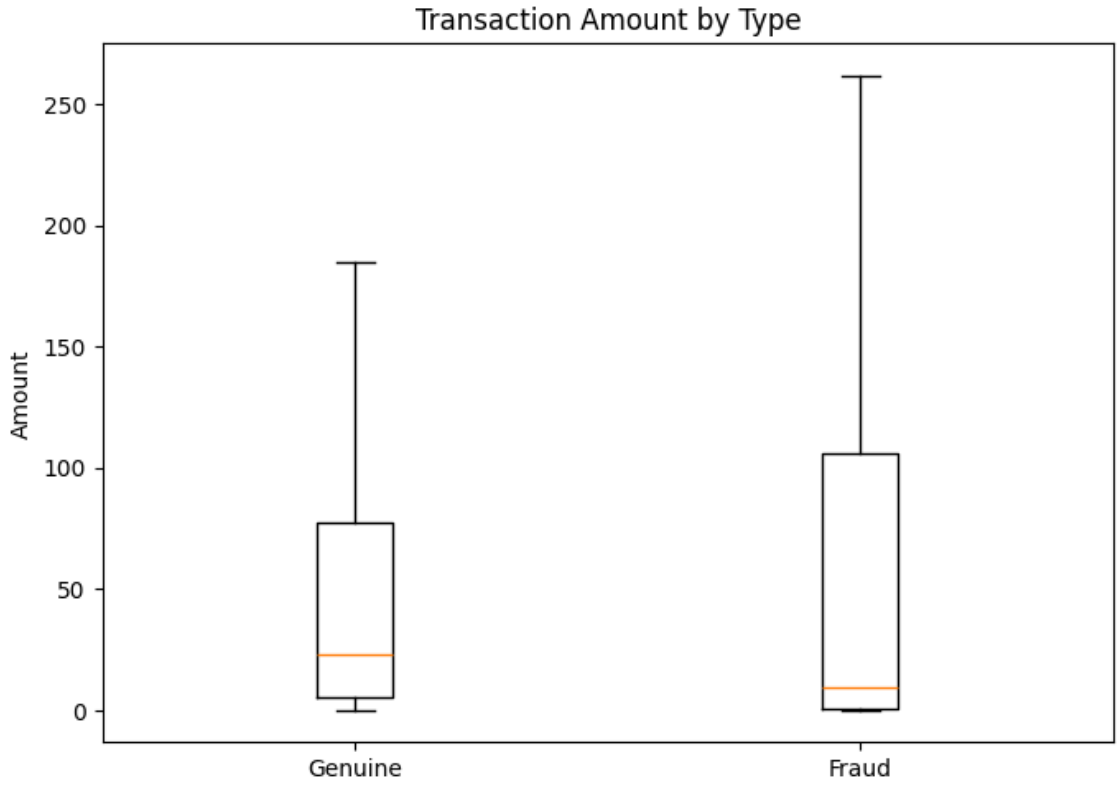


Figure 3: Transaction amount by type (box plot, outliers hidden)

Figure 3 compares the Amount of genuine and fraud transactions. Approximate readings:

Measure	Genuine	Fraud
Median	about 23	about 9
Middle half (first to third quartile)	about 5 to 77	about 1 to 106

Measure	Genuine	Fraud
Upper whisker	about 185	about 262

Fraud transactions have a lower median but a wider spread, with many very small amounts and some large ones. Both distributions are strongly right-skewed (the outliers are hidden in this plot, and amounts of several thousand exist, see Figure 12). Amount alone therefore cannot separate the classes.

7.4 What the EDA Implies for Modelling

- The strong class imbalance means precision, recall and F1 must be reported in addition to accuracy.
- Many V features are clearly associated with Class, so even a linear model such as Logistic Regression is expected to perform well.
- Time and Amount are on a very different scale from the V features, which supports the use of StandardScaler.
- Amount is skewed with large outliers, which will make regression on Amount difficult (Section 11.8).

8. Methodology

8.1 Overall Workflow

The project follows the standard pipeline shown in Figure 4. The raw file is cleaned and reduced to a working sample, explored with graphs, and then used for three tasks: classification of Class, regression of Amount, and clustering of the V features. The results of all three tasks are compared in Chapter 11.

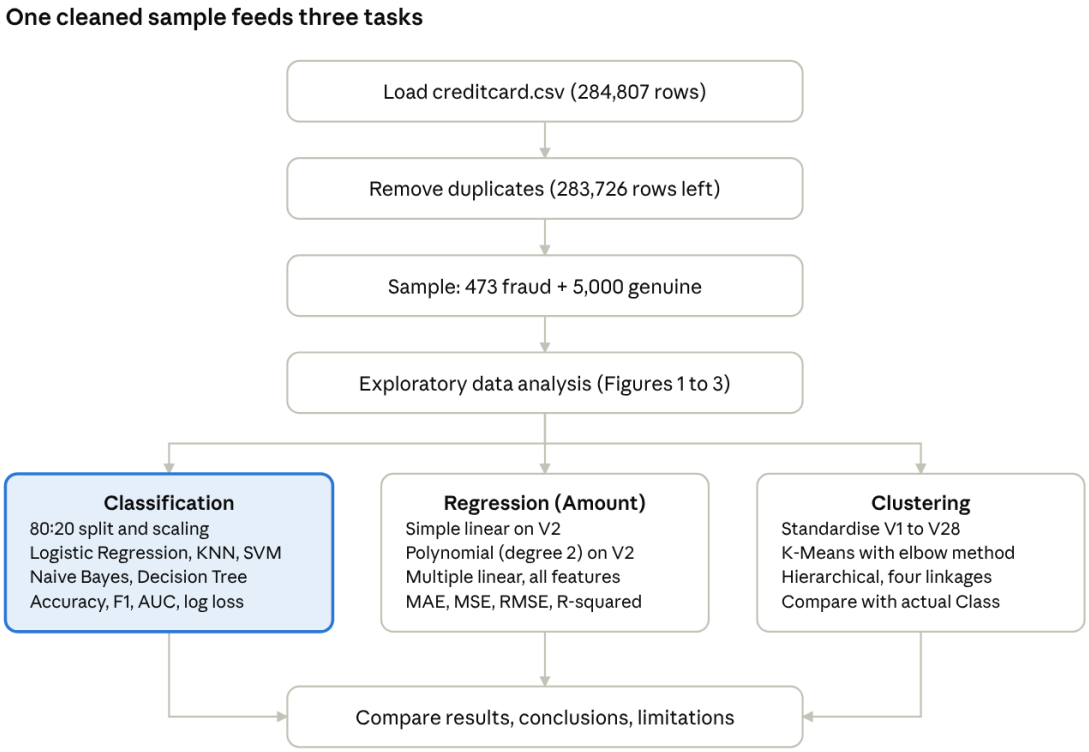


Figure 4: Workflow of the credit card fraud detection project

8.2 Experimental Design

- **Task 1 (classification):** two-class prediction of Class (genuine or fraud).
- **Task 2 (regression):** prediction of the numerical Amount.
- **Task 3 (clustering):** grouping of transactions without using Class.
- **Data:** the 5,473-row sample of the cleaned dataset (Chapter 6).
- **Classifiers:** Logistic Regression, KNN, Gaussian Naive Bayes, Decision Tree and SVM.
- **Evaluation of classifiers:** a held-out test set of 1,095 transactions and the measures defined in Section 8.3.
- **Comparison:** all classifiers are compared on the same measures in one table and four graphs (Chapter 11).

8.3 Evaluation Measures

Classification. For a two-class problem the confusion matrix has four cells. Fraud is treated as the positive class: TP is a fraud transaction predicted as fraud, FP is a

genuine transaction predicted as fraud (a false alarm), FN is a fraud transaction predicted as genuine (a missed fraud) and TN is a genuine transaction predicted as genuine.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}, \qquad \text{Precision} = \frac{TP}{TP + FP}, \qquad \text{Recall} = \frac{TP}{TP + FN}$$

$$F_1 = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}, \qquad \text{Log loss} = -\frac{1}{N} \sum_{i=1}^{N} \left[y_i \ln p_i + (1 - y_i) \ln (1 - p_i) \right]$$

The ROC curve plots the true positive rate against the false positive rate for every possible threshold, and ROC AUC is the area under it; 0.5 is random guessing and 1.0 is perfect separation. Log loss uses the predicted probability p of fraud and punishes confident wrong predictions heavily, so a lower value is better.

Measure	Question it answers	Weakness
Accuracy	Of all transactions, what fraction was classified correctly?	Hides poor performance on the rare class
Precision	When the model says “fraud”, how often is it right?	Ignores frauds that were missed
Recall	Of all real frauds, how many were found?	Ignores false alarms
F1-score	Balance of precision and recall (harmonic mean)	Ignores true negatives
ROC AUC	How well does the model rank frauds above genuine transactions?	Can look optimistic when the positive class is rare
Log loss	How good are the predicted probabilities?	Hard to read without a comparison

Measure	Question it answers	Weakness
Confusion matrix	Which errors does the model make?	Needs reading cell by cell

Precision, recall and F1 are reported for the fraud class (class 1), which is the default of the scikit-learn functions for a two-class problem. Because fraud is the class of interest, these three values are more informative than accuracy.

Regression. With actual values y and predictions ŷ for n test records:

```
\text{MAE} = \frac{1}{n} \sum |y_i - \hat{y}_i|, \quad \text{MSE} = \frac{1}{n} \sum (y_i - \hat{y}_i)^2, \quad \text{RMSE} = \sqrt{\text{MSE}}, \quad R^2 = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}
```

MAE and RMSE are in the same unit as Amount; RMSE punishes large errors more than MAE. R^2 is the fraction of the variation in Amount that the model explains (1 is perfect, 0 is no better than predicting the mean).

Clustering. K-Means minimises the within-cluster sum of squares (WCSS, called inertia in scikit-learn), the sum of squared distances from every point to the centre of its cluster. The quality of the clusters is also checked by cross-tabulating them against the real Class.

8.4 Reproducibility and Fairness of Comparison

A fair comparison requires that every model sees the same training data and is judged on the same test data with the same measures. In the program every random step uses a fixed seed (42 for sampling, shuffling, splitting, Decision Tree, SVM and K-Means; 1 or 2 or 3 for the initialisation comparison; 1 for the dendrogram sample and the sample predictions), all five classifiers are trained on the same scaled training data and tested on the same scaled test data, and all measures are computed by the same scikit-learn functions. Running the program again on the same file therefore gives the same numbers.

9. Algorithms Used

This chapter explains each algorithm in plain language: the main idea, the key formula, strengths and weaknesses, and the settings used in this project.

9.1 Logistic Regression

Idea. Logistic regression computes a weighted sum of the features and converts it into a probability with the sigmoid function; the weights are found by maximising the likelihood of the training labels [5].

$$P(y = 1 \mid x) = \frac{1}{1 + e^{-(w \cdot x + b)}}$$

Strengths: fast, stable, easy to interpret. **Weaknesses:** the boundary is linear. **Settings:** max_iter = 1000, other settings at their defaults.

9.2 k-Nearest Neighbours (KNN)

Idea. To classify a transaction, KNN finds the k closest transactions in the training set and gives it the class that most of them have [3]. There is no training phase; the data are simply stored (lazy learning). Closeness is measured by the Euclidean distance.

$$d(x, x') = \sqrt{\sum_{j} (x_j - x'_j)^2}$$

Strengths: simple, follows irregular boundaries. **Weaknesses:** slow on large data, sensitive to feature scale and to irrelevant features. **Settings:** k = 5 (n_neighbors = 5).

9.3 Gaussian Naive Bayes

Idea. Naive Bayes uses Bayes' theorem to compute the probability of each class given the features, under the "naive" assumption that the features are independent within each class. In the Gaussian version each feature is described by a normal distribution per class [14].

$$P(c \mid x) \propto P(c) \prod_{j} P(x_j \mid c)$$

Strengths: very fast, needs little data. **Weaknesses:** the independence and normality assumptions are only approximate, and the probabilities it outputs are often over-confident [6]. **Settings:** defaults.

9.4 Decision Tree

Idea. A decision tree asks a sequence of yes/no questions about the features (for example, "is V14 smaller than a threshold?"). Each question splits the transactions into two groups, chosen to make the groups as pure as possible, measured here by Gini impurity [4].

$$\text{Gini} = 1 - \sum_{k} p_k^2$$

Strengths: easy to explain as rules, needs no scaling, captures non-linear patterns. **Weaknesses:** an unrestricted tree over-fits [4]. **Settings:** max_depth = 5 and random_state = 42; the depth limit keeps the tree small.

9.5 Support Vector Machine (SVM)

Idea. An SVM finds the boundary that separates the classes with the widest margin; the points closest to the boundary are the support vectors [2]. A soft margin tolerates some misclassified points (controlled by the penalty C), and a kernel allows a curved boundary.

```
f(x) = \text{sign}\left( \sum_i \alpha_i y_i K(x_i, x) + b \right)
```

Strengths: effective on medium-sized data with scaled features. **Weaknesses:** needs scaling, sensitive to the kernel and C, slow on large data. **Settings:** RBF kernel, default C = 1.0, probability = True (so that ROC AUC and log loss can be computed), random_state = 42.

Algorithm	Boundary type	Needs scaling?	Interpretability	Main risk
Logistic Regression	Linear	Helpful	High	Under-fitting
KNN	Local, flexible	Yes	Medium	Scale and k sensitivity, slow prediction
Gaussian Naive Bayes	Probabilistic	No	Medium	Independence assumption, poor probabilities
Decision Tree	Axis-aligned splits	No	High	Over-fitting
SVM	Linear or kernel-based	Yes	Low	Parameter sensitivity

9.6 Regression: Simple, Polynomial and Multiple Linear

Ordinary least squares (OLS). Linear regression chooses the coefficients that minimise the sum of squared differences between the actual and predicted values [15].

$$\text{Simple: } \hat{y} = b_0 + b_1 x \qquad \text{Polynomial (degree 2): } \hat{y} = b_0 + b_1 x + b_2 x^2 \qquad \text{Multiple: } \hat{y} = b_0 + \sum_{j} b_j x_j$$

In this project the target is the transaction **Amount**. The simple and polynomial models use one feature, **V2**, which is the feature most correlated with Amount (correlation −0.4154, Chapter 11). The multiple model uses Time and V1 to V28. The polynomial model adds the squared term of V2 so that the fitted line can curve.

9.7 K-Means Clustering

Idea. K-Means divides the data into K groups. It places K centres, assigns every point to its nearest centre, moves each centre to the mean of its points, and repeats until nothing changes [12]. The aim is to minimise the within-cluster sum of squares (WCSS).

$$\text{WCSS} = \sum_{k=1}^{K} \sum_{x \in C_k} \lVert x - \mu_k \rVert^2$$

Random initialisation trap. The final result depends on where the centres start. With a bad random start the algorithm can settle in a poor local minimum. k-means++ spreads the starting centres apart and usually gives better and more stable results [13]; running the algorithm several times (n_init) and keeping the best run also helps. The program compares random and k-means++ starts in Section 11.9.

Choosing K. The elbow method plots WCSS against K for K = 1 to 10 and looks for the point after which the curve flattens. **Settings:** init = k-means++, n_init = 10, random_state = 42; K = 2 was used for the final clusters because the data have two real classes and the elbow plot shows its largest drop at K = 2.

9.8 Hierarchical (Agglomerative) Clustering

Idea. Every point starts as its own cluster, and the two closest clusters are merged repeatedly until one cluster is left. The history of the merges is drawn as a dendrogram. How "close" two clusters are depends on the linkage [14]:

Linkage	Distance between two clusters	Typical behaviour
Single	Smallest distance between any two points, one from each cluster	Can chain points into long, thin clusters
Complete	Largest distance between any two points, one from each cluster	Tends to give compact clusters of similar size
Average	Mean of all pairwise distances between the two clusters	A compromise between single and complete
Centroid	Distance between the two cluster centres	Similar to average; can produce reversals in the dendrogram

Settings: AgglomerativeClustering with n_clusters = 2 and average linkage was run on all 5,473 samples; dendrograms of all four linkages were drawn for a smaller sample of 150 transactions (100 genuine and 50 fraud) so that they are readable.

10. Implementation and Code Explanation

The complete program is a single Python file (final.py, run in IDLE with Python 3.11.9). The full source code is in Appendix B. This chapter explains it stage by stage in simple language.

10.1 Tools and Libraries

Library	Role in the project
pandas [17]	Loading the CSV file, cleaning, sampling, grouping, cross-tabulation
NumPy [18]	Numerical arrays, square root for RMSE, sorting for the regression plots

Library	Role in the project
Matplotlib [19]	Drawing all 14 graphs
scikit-learn [1]	StandardScaler, train_test_split, the five classifiers, LinearRegression, PolynomialFeatures, KMeans, AgglomerativeClustering and all metric functions
SciPy [20]	linkage and dendrogram functions for the hierarchical-clustering dendrograms

10.2 Stage-by-Stage Explanation

Stage 1: Importing libraries and loading data. The libraries are imported and creditcard.csv is read into a pandas DataFrame (a table with named columns). The program prints the number of records (284,807) and columns (31), the first five rows and the summary statistics (mean, standard deviation, minimum and maximum) of the first columns.

Stage 2: Cleaning. drop_duplicates() removes repeated rows, dropna() removes rows with missing values, and the Class column is converted to integers. The program prints the number of missing values (0), the shape after cleaning (283,726 × 31) and the number of genuine (283,253) and fraud (473) transactions. A folder named project_outputs is created for all results.

Stage 3: Sampling. All fraud rows are kept, 5,000 genuine rows are taken with sample(5000, random_state=42), and the two parts are joined and shuffled. The sample has 5,473 rows.

Stage 4: Features, target, split and scaling. The 30 input columns (Time, Amount, V1 to V28) form X and Class forms y. train_test_split(test_size=0.20, random_state=42, stratify=y) gives 4,378 training and 1,095 test records. A StandardScaler is fitted on the training data with fit_transform and applied to the test data with transform.

Stage 5: Training and evaluating the five classifiers. The five models are kept in a dictionary. A loop trains each one with fit(), predicts the classes with predict() and the fraud probability with predict_proba(). It then computes accuracy, precision, recall, F1-score, ROC AUC and log loss. The results are collected in a table sorted

by F1-score and saved as model_comparison.csv. The model with the highest F1 (SVM) is chosen as the “best” model, and its classification report is printed.

Stage 6: Classification graphs. The program draws the class distribution (Figure 1), the top 12 correlations with Class (Figure 2), the Amount box plot (Figure 3), the accuracy, F1 and log-loss bar charts (Figures 6 to 8), the ROC curves of all five models (Figure 9) and the confusion matrix of the best model (Figure 10).

Stage 7: Regression. The feature most correlated with Amount is found with corr() and idxmax() on absolute values (V2). A LinearRegression model is fitted on V2 (simple), another on V2 and its square created by PolynomialFeatures(degree=2) (polynomial), and a third on Time and V1 to V28 (multiple). A small function computes MAE, MSE, RMSE and R^2 for each, and the table is saved as regression_comparison.csv. Three graphs are drawn (Figures 12 to 14).

Stage 8: Clustering. The V1 to V28 columns are standardised. KMeans is run for K = 1 to 10 and the inertia is plotted (elbow method, Figure 16). Random and k-means++ initialisation are compared with three seeds. K-Means with K = 2 is run, and the clusters are cross-tabulated with Class and plotted on V1 and V2 (Figure 17). AgglomerativeClustering with average linkage and two clusters is run on the whole sample, and four dendrograms (single, complete, average, centroid) are drawn from a 150-transaction sample with SciPy (Figure 18).

Stage 9: Saving and sample predictions. The test records with their actual class, predicted class and fraud probability are saved as test_predictions.csv. Finally, five random test transactions are scored with the best model and printed together with the actual class.

Files created in the project_outputs folder: 14 PNG graphs (01_class_distribution.png to 14_dendrograms.png), model_comparison.csv, regression_comparison.csv and test_predictions.csv.

10.3 Typical Questions on the Code (Viva Preparation)

Question	Short answer
Why did you take only 5,473 rows?	KNN and SVM are slow on 283,726 rows. All 473 fraud cases were kept and 5,000

Question	Short answer
	genuine ones sampled at random. The side effect is that the fraud share rises to 8.64%.
Why StandardScaler?	Time and Amount have very different ranges from V1 to V28. KNN and SVM depend on distances or margins, so large-range features would dominate.
Why fit the scaler on training data only?	To avoid leaking information from the test set into training.
Why stratify the split?	So that the training and test sets both keep the same share of fraud (95 fraud cases in the test set).
Why not use accuracy only?	Always predicting “genuine” already gives 91.3% accuracy on the test set. Precision, recall and F1 show how fraud is handled.
What is the difference between fit() and predict()?	fit() learns from the training data; predict() applies what was learned to new data.
What does the confusion matrix show?	For each true class, how many were predicted as each class. The diagonal holds the correct predictions.
What is log loss?	A measure of the quality of predicted probabilities. Confident wrong predictions are punished heavily; lower is better.
What is OLS?	Ordinary least squares: choosing regression coefficients that minimise the sum of squared errors.

Question	Short answer
What do MAE, MSE, RMSE and R^2 mean?	Average absolute error, average squared error, its square root (in the unit of Amount), and the fraction of variation explained.
How did you choose K in K-Means?	With the elbow method (WCSS against K). The biggest drop is from K = 1 to K = 2.
What is the random initialisation trap?	A bad random start can trap K-Means in a poor local minimum. k-means++ and several restarts (n_init) reduce the risk.
Which model was best, and why is that claim limited?	SVM had the best accuracy, F1 and AUC, but Logistic Regression is almost identical, and only one split was used with 95 fraud cases in the test set.

11. Results and Discussion

All numerical results in this chapter are taken from the output of the program (screenshots in Figures 5, 11 and 15) and from the graphs it produced. Where a quantity was derived by simple arithmetic from those results, it is stated to be derived. Values read from a graph are approximate.

11.1 Model Comparison Table

[Insert Figure 5 here: IDLE screenshot of the MODEL COMPARISON table]

Figure 5: Program output – model comparison on the test set (1,095 transactions)

Model	Accuracy	Precision	Recall	F1-score	ROC AUC	Log loss
SVM	0.9817	0.9870	0.8000	0.8837	0.9847	0.0767

Model	Accuracy	Precision	Recall	F1-score	ROC AUC	Log loss
Logistic Regression	0.9808	0.9512	0.8211	0.8814	0.9826	0.0671
Decision Tree	0.9808	0.9625	0.8105	0.8800	0.9494	0.1718
KNN	0.9799	0.9620	0.8000	0.8736	0.9332	0.4474
Naive Bayes	0.9644	0.7800	0.8211	0.8000	0.9713	0.8224

The table is sorted by F1-score, as in the program. “Naive Bayes” is the Gaussian Naive Bayes classifier. Precision, recall and F1 are for the fraud class; for log loss, lower is better.

11.2 Model Comparison Graphs

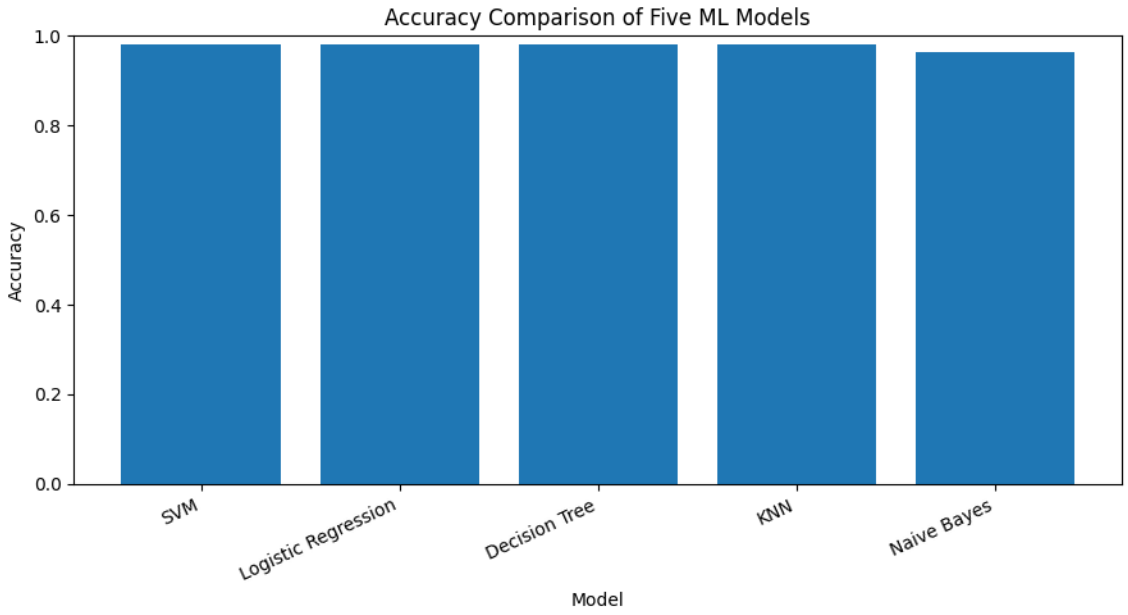


Figure 6: Accuracy comparison of the five models

Figure 6: SVM, Logistic Regression, Decision Tree and KNN are all at about 0.98, and Naive Bayes is slightly lower at about 0.96. Because the axis starts at 0, the differences look very small.

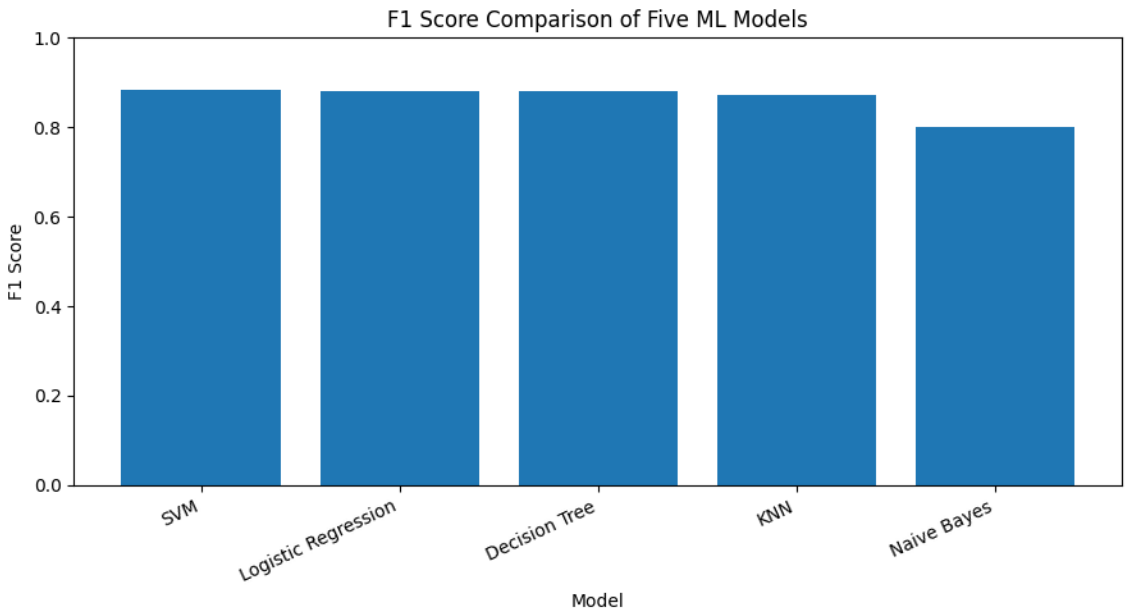


Figure 7: F1-score comparison of the five models

Figure 7: SVM, Logistic Regression and Decision Tree are all at about 0.88, KNN is at about 0.87, and Naive Bayes is clearly lower at 0.80.

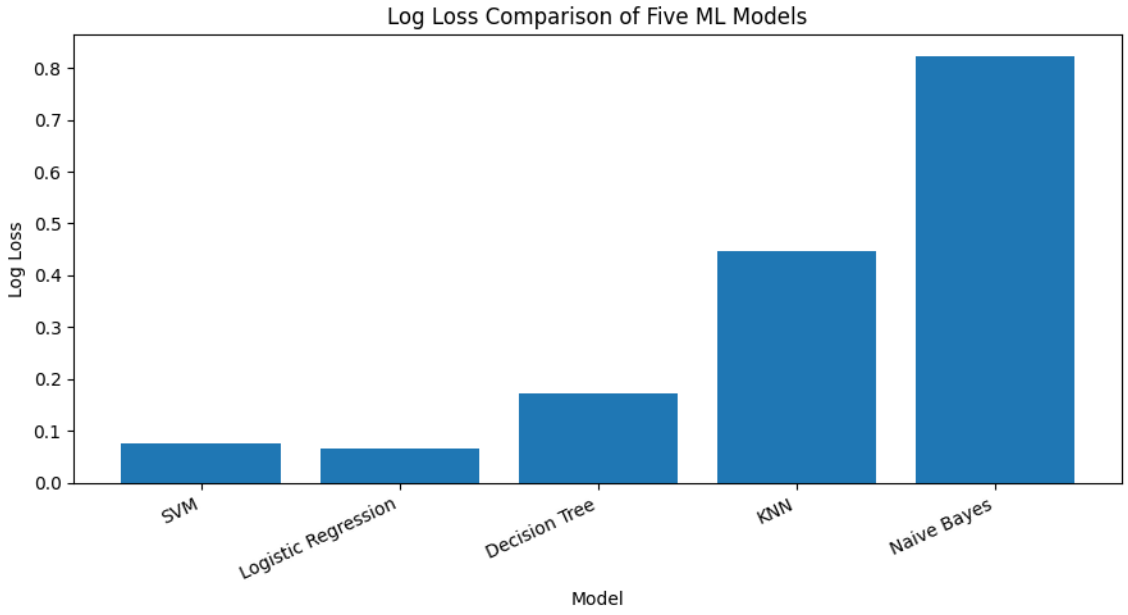


Figure 8: Log loss comparison of the five models (lower is better)

Figure 8: Logistic Regression (about 0.07) and SVM (about 0.08) have the lowest log loss, Decision Tree is about 0.17, KNN about 0.45 and Naive Bayes about 0.82.

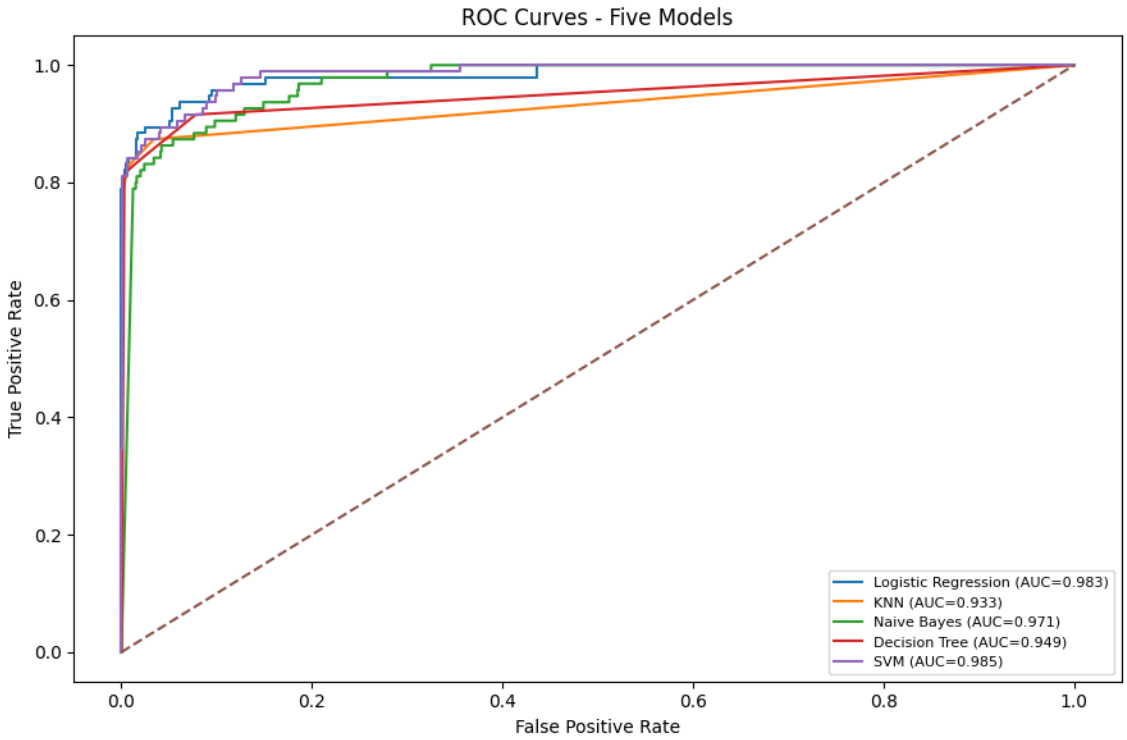


Figure 9: ROC curves of the five models

Figure 9: All five curves rise steeply towards the top-left corner and are far above the diagonal (random guessing). The AUC values in the legend are SVM 0.985, Logistic Regression 0.983, Naive Bayes 0.971, Decision Tree 0.949 and KNN 0.933. The KNN and Decision Tree curves have long straight segments because they output only a few different probability values.

11.3 Ranking and Observations

Rank	Accuracy	Precision	Recall	F1-score	ROC AUC	Log loss (lowest best)
1	SVM (0.9817)	SVM (0.9870)	Logistic Regression and Naive Bayes (0.8211)	SVM (0.8837)	SVM (0.9847)	Logistic Regression (0.0671)

Rank	Accuracy	Precision	Recall	F1-score	ROC AUC	Log loss (lowest best)
2	Logistic Regression and Decision Tree (0.9808)	Decision Tree (0.9625)	(tie above)	Logistic Regression (0.8814)	Logistic Regression (0.9826)	SVM (0.0767)
3	(tie above)	KNN (0.9620)	Decision Tree (0.8105)	Decision Tree (0.8800)	Naive Bayes (0.9713)	Decision Tree (0.1718)
4	KNN (0.9799)	Logistic Regression (0.9512)	SVM and KNN (0.8000)	KNN (0.8736)	Decision Tree (0.9494)	KNN (0.4474)
5	Naive Bayes (0.9644)	Naive Bayes (0.7800)	(tie above)	Naive Bayes (0.8000)	KNN (0.9332)	Naive Bayes (0.8224)

The following observations follow directly from the numbers:

- **Top group.** SVM and Logistic Regression are very close. Their accuracy differs by 0.0009, which is about one transaction out of 1,095, and their F1-scores differ by 0.0023. With one train-test split, such tiny differences do not prove that one model is better.
- **SVM** leads on accuracy, precision, F1 and ROC AUC. **Logistic Regression** leads on log loss and is tied for the highest recall.
- **Decision Tree and KNN** are close behind in accuracy (0.9808 and 0.9799) and F1 (0.8800 and 0.8736), but KNN has a much higher log loss (0.4474, about 6.7 times that of Logistic Regression), and its ROC AUC (0.9332) is the lowest.
- **Naive Bayes** is the weakest on accuracy (0.9644), precision (0.7800), F1 (0.8000) and log loss (0.8224, about 12 times that of Logistic Regression), although its recall (0.8211) is the joint highest.

- **No model exceeds a recall of 0.8211**, so every model misses at least 17 of the 95 fraud transactions in the test set (derived, Section 11.6).

11.4 SVM Confusion Matrix and Classification Report

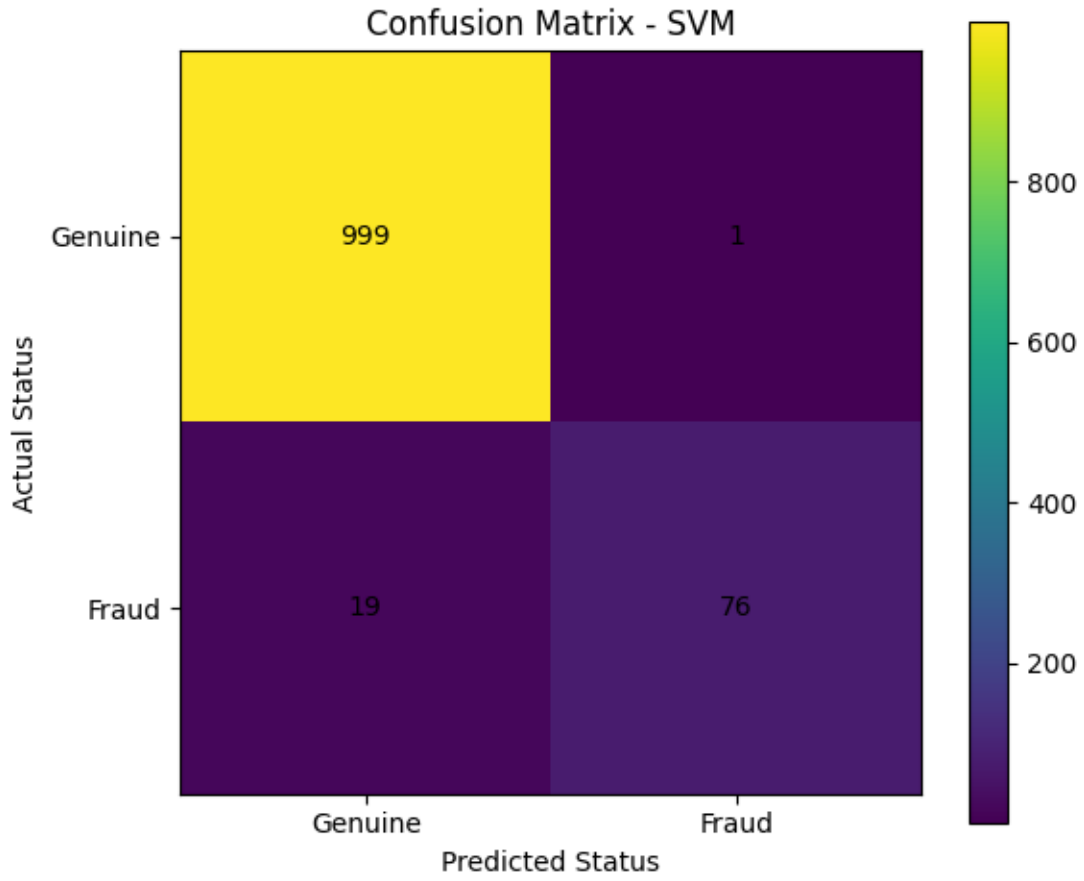


Figure 10: Confusion matrix of the SVM (best model by F1-score) on the test set

Rows are the true classes and columns the predicted classes. The same counts are shown as a table, with totals.

True class \ Predicted	Genuine	Fraud	Row total
Genuine	999	1	1,000
Fraud	19	76	95
Column total	1,018	77	1,095

Check. The correct predictions are the diagonal: 999 + 76 = 1,075 out of 1,095, and 1,075 / 1,095 = 0.9817, which matches the SVM accuracy in Section 11.1.

The program also printed the classification report of the SVM:

Class	Precision	Recall	F1-score	Support
Genuine	0.98	1.00	0.99	1,000
Fraud	0.99	0.80	0.88	95
Accuracy			0.98	1,095
Macro average	0.98	0.90	0.94	1,095
Weighted average	0.98	0.98	0.98	1,095

Observations from the confusion matrix:

- **Genuine transactions.** 999 of 1,000 were classified correctly; only one was wrongly flagged as fraud.
- **Fraud transactions.** 76 of 95 were found (recall 0.80); 19 were missed.
- **Precision.** The SVM flagged 77 transactions as fraud and 76 of them were really fraud (precision 0.987).
- **Where the errors come from.** There are 20 wrong predictions in total (1 + 19), and 19 of them (95%) are missed frauds (derived). The model is therefore very careful about false alarms but misses about one fraud in five.

11.5 Per-Class Metrics for the SVM (Derived)

From the counts above, the per-class values can be calculated with the formulas of Section 8.3. These are derived values, not additional experimental output.

Class	Precision	Recall	F1-score	Support
Genuine	0.981	0.999	0.990	1,000
Fraud	0.987	0.800	0.884	95
Macro average	0.984	0.900	0.937	1,095

The macro averages equal the macro-average row printed by the program (0.98, 0.90, 0.94) when rounded to two decimals, so the report and the confusion matrix are consistent. The fraud class has the lower recall and F1, and it is the class that matters most.

11.6 Confusion-Matrix Counts of All Five Models (Derived)

The program printed the confusion matrix only for the best model. The counts for the other models can be recovered exactly from the printed recall and precision and the test-set sizes (95 fraud and 1,000 genuine): TP = recall × 95, FP = TP / precision − TP, FN = 95 − TP and TN = 1,000 − FP. They reproduce every reported accuracy (for example, (78 + 996) / 1,095 = 0.9808 for Logistic Regression).

Model	TP (fraud found)	FN (fraud missed)	FP (false alarms)	TN	Total errors
SVM	76	19	1	999	20
Logistic Regression	78	17	4	996	21
Decision Tree	77	18	3	997	21
KNN	76	19	3	997	22
Naive Bayes	78	17	22	978	39

The five models miss almost the same number of frauds (17 to 19). What separates them is the number of false alarms: only 1 for the SVM, but 22 for Naive Bayes, which is why Naive Bayes has the lowest precision (0.78) and the lowest accuracy.

11.7 Majority-Class Baseline and the Effect of Sampling

Baseline. The test set has 1,000 genuine transactions out of 1,095, so predicting "genuine" for everything gives an accuracy of 91.3% (derived). The accuracies of the five models (0.9644 to 0.9817) are all above this level, by 5.1 to 6.9 percentage points, so every model has learned useful information from the features. The gain looks small in accuracy, but it corresponds to finding about 80% of the fraud cases, which a constant rule never finds.

Effect of the sampling. The sample contains 8.64% fraud, whereas the cleaned data contain only 0.17%. The results of this report therefore describe performance on a rebalanced sample, and they should not be read as performance on real traffic:

- Accuracy on the full data would have a baseline of 99.83% (283,253 / 283,726), not 91.3%, so accuracies from the sample cannot be compared with accuracies on the full data.
- Precision falls when fraud becomes rarer, because the same false-alarm rate produces many more false alarms per fraud. As an illustration (derived, not measured): if the SVM kept its recall of 0.80 and its false-alarm rate of 1 in 1,000, then among 100,000 real transactions it would find about 133 of about 167 frauds and raise about 100 false alarms, giving a precision of about 0.57 instead of 0.987. The false-alarm rate itself rests on a single error and is therefore very uncertain.
- Under-sampling also changes predicted probabilities [10], so log loss and probability values are specific to this sample.

Testing on the full, unsampled data is listed as future work.

11.8 Regression Results (Predicting Transaction Amount)

[Insert Figure 11 here: IDLE screenshot of the REGRESSION output]

Figure 11: Program output – regression results

The feature most correlated with Amount is **V2** (correlation −0.4154). Three models were fitted on 80% of the sample and evaluated on the remaining 1,095 records.

Model	MAE	MSE	RMSE	R^2 score
Simple Linear (V2)	114.0089	66,145.5142	257.1877	0.2456
Polynomial, degree 2 (V2)	106.8985	58,019.3905	240.8721	0.3383
Multiple Linear (Time, V1–V28)	37.6475	8,943.6042	94.5706	0.8980

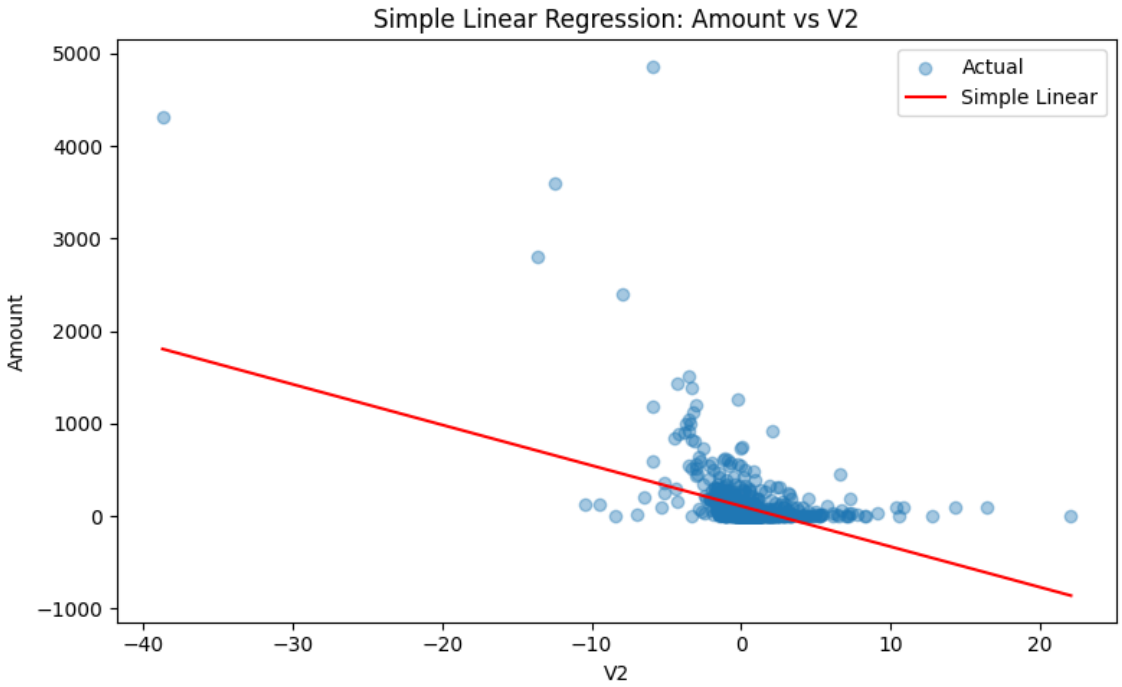


Figure 12: Simple linear regression, Amount against V2

Figure 12: most points lie where V2 is between about −5 and 5 and Amount is below about 1,000, and a few points with very large amounts (about 2,400 to 4,900) have V2 between about −39 and −6. The fitted line slopes downwards and falls below zero for large V2 (about −850 at V2 = 22), which is impossible for a real amount.

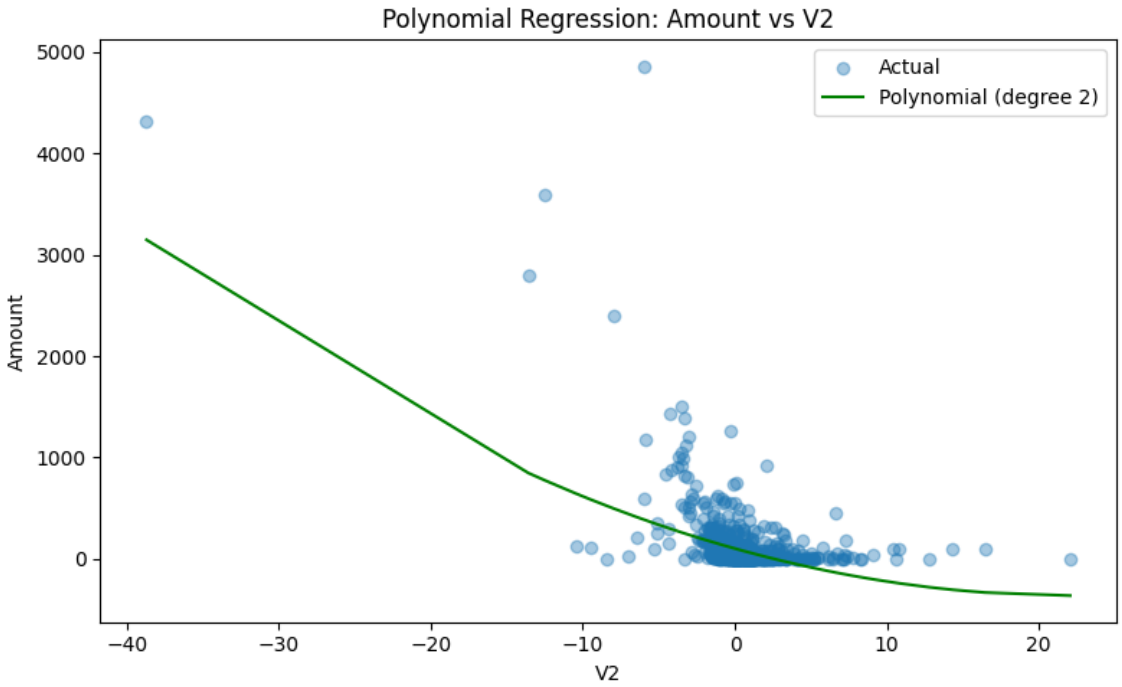


Figure 13: Polynomial regression (degree 2), Amount against V2

Figure 13: the curve bends and follows the large-amount points on the left better than the straight line (about 3,150 at V2 = −38), but it also becomes negative for large V2 (about −350 at V2 = 22).

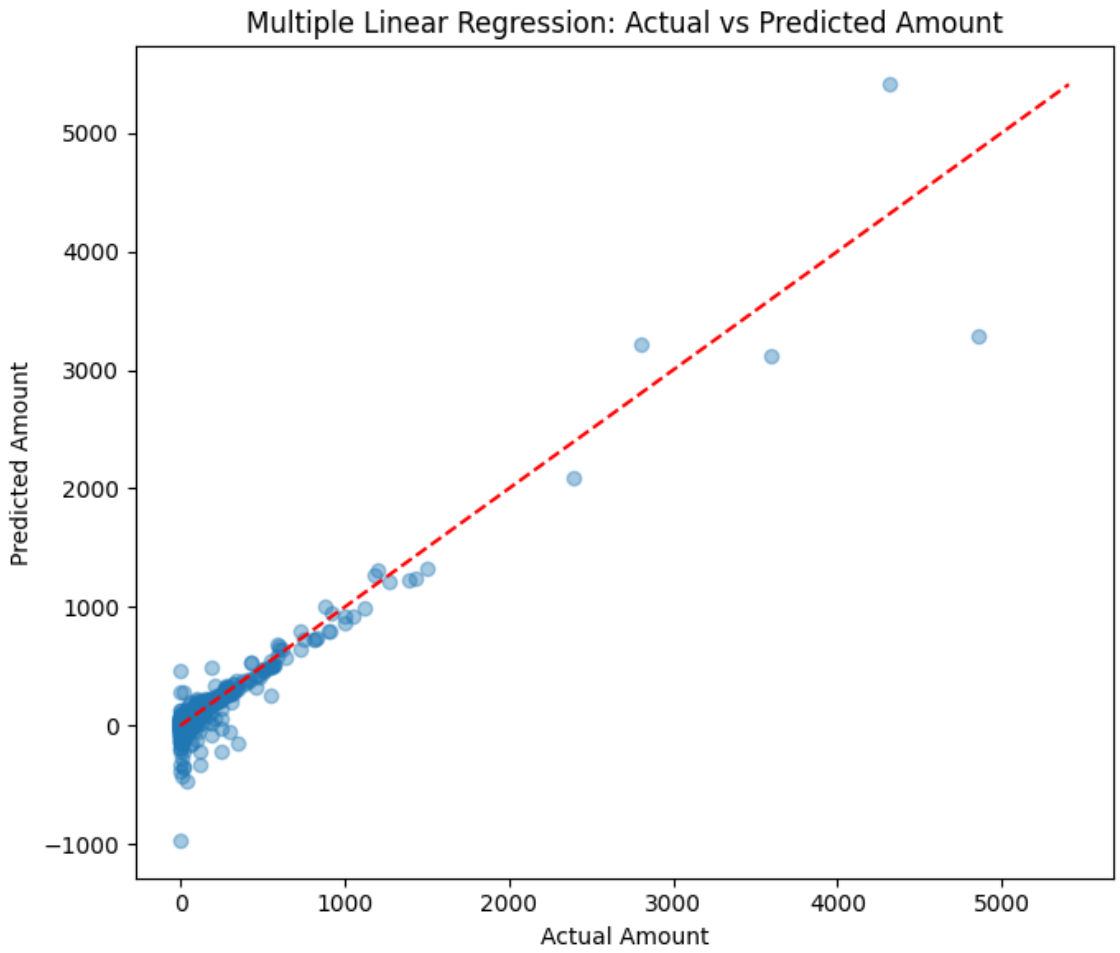


Figure 14: Multiple linear regression, actual against predicted Amount

Figure 14: the points follow the dashed diagonal (perfect prediction) closely for large amounts, although the point at an actual amount of about 4,300 is over-predicted (about 5,400) and the point at about 4,850 is under-predicted (about 3,300). For small amounts the predictions are scattered and some are negative (down to about −970).

Observations:

- Using all 29 inputs instead of V2 alone cuts the MAE from 114.01 to 37.65 (−67%) and the RMSE from 257.19 to 94.57 (−63%) (derived). R^2 rises from 0.2456 to 0.8980, i.e. the multiple model explains about 90% of the variation of Amount in the test records.
- Adding the squared term improves the single-feature model only moderately (R^2 0.2456 to 0.3383), so one feature is not enough.

- RMSE is more than twice the MAE for every model, which shows that a few very large amounts produce large errors.
- The V columns are PCA components computed by the data providers from confidential attributes. A high R^2 therefore shows a statistical relationship, not a business explanation, and the components may already contain information related to Amount; this cannot be checked from the data.

11.9 Clustering Results

[Insert Figure 15 here: IDLE screenshot of the CLUSTERING output]

Figure 15: Program output – initialisation comparison and cluster tables

K-Means: choosing K (elbow method).

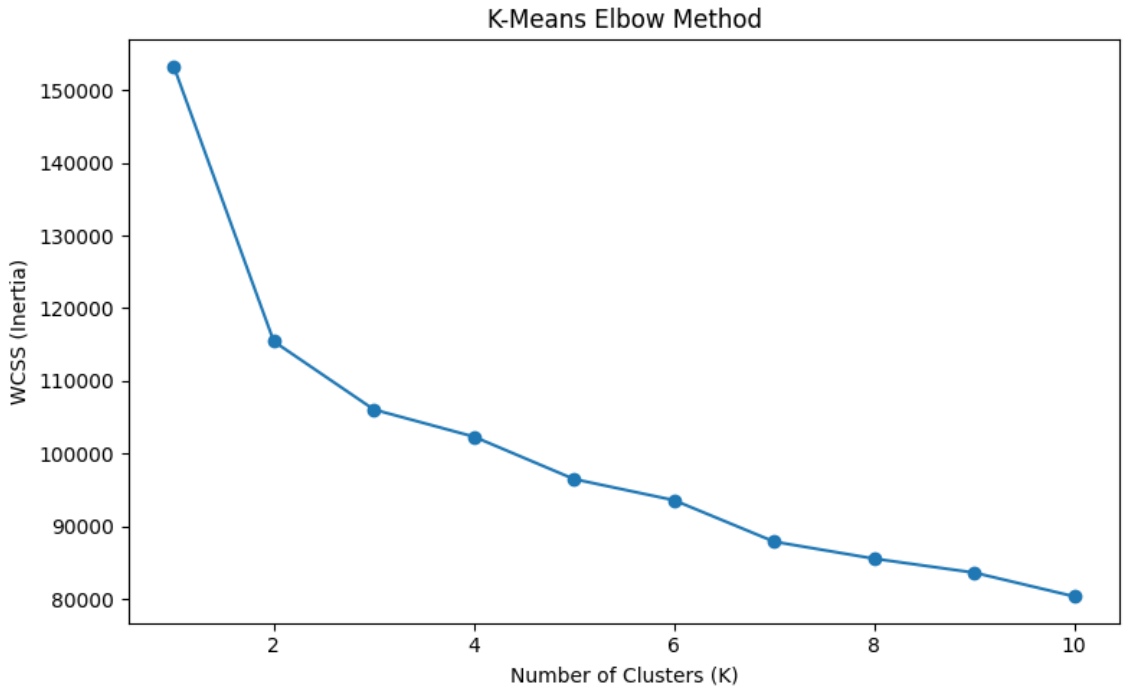


Figure 16: K-Means elbow method (WCSS against K)

Figure 16: the WCSS falls from about 153,000 at K = 1 to about 115,500 at K = 2 (a drop of about 25%), then from about 106,000 at K = 3 to about 80,500 at K = 10 in small steps of roughly 2,000 to 6,000 each. The biggest bend is therefore at K = 2, which fits the two real classes, but the curve has no sharp elbow beyond it.

Random initialisation compared with k-means++ (K = 3, one run each, inertia):

Seed	Random initialisation	k-means++
1	106,075.1	112,072.4
2	111,885.6	106,075.4
3	111,840.4	111,842.2

All six runs ended at one of two levels: about 106,075 or about 111,840 to 112,070. A single run can therefore get stuck in a worse solution (about 5.7% higher inertia), whichever initialisation is used. In this small test k-means++ was better for seed 2, worse for seed 1 and equal for seed 3, so no general winner can be named from three runs. The final model uses n_init = 10, i.e. ten starts of which the best is kept, to reduce the risk of this trap.

K-Means with K = 2 compared with the real classes:

K-Means cluster	Genuine	Fraud
0	5,000	282
1	0	191

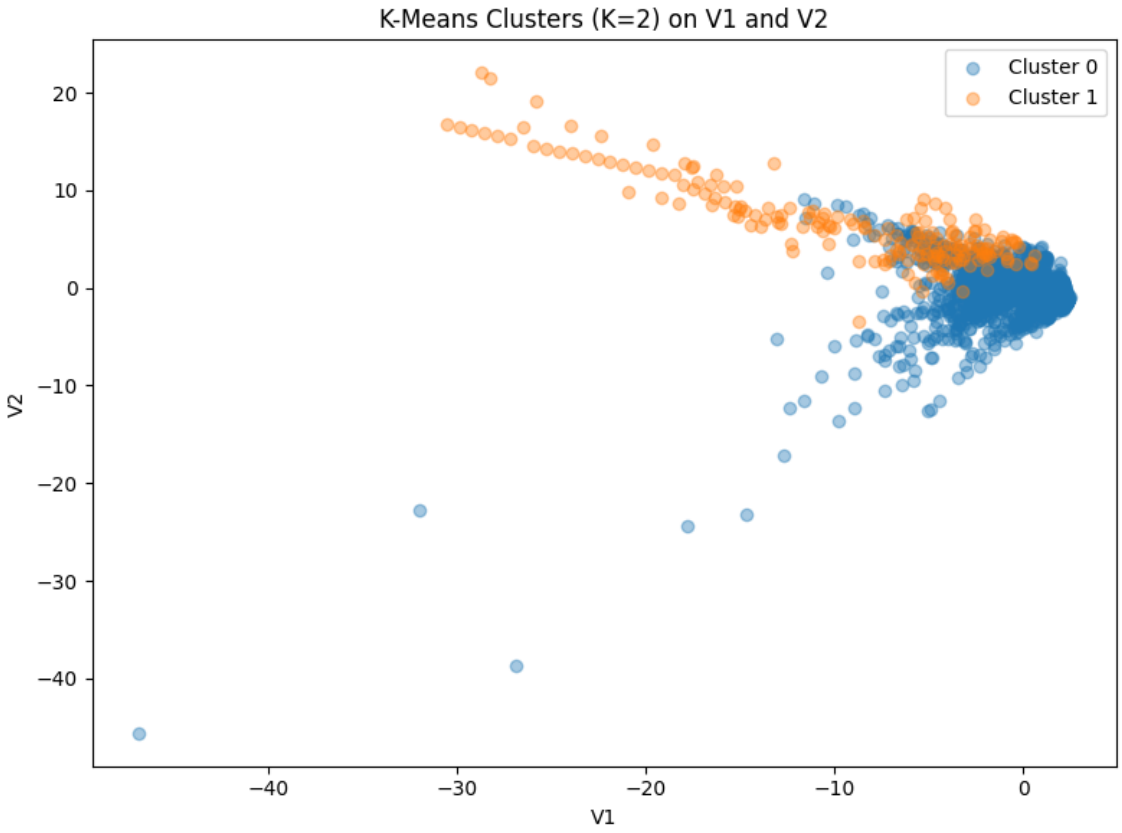


Figure 17: K-Means clusters (K = 2) plotted on V1 and V2

Figure 17: cluster 0 is the dense mass around the origin and also contains scattered extreme points; cluster 1 forms a thin band at low V1 (below about −5) and high V2 (up to about 22).

Cluster 1 contains **only fraud** (191 transactions, precision 100%), but it holds only 191 of 473 fraud cases (40.4%, derived). Cluster 0 holds 5,000 genuine and 282 fraud transactions, so it is 94.7% genuine (derived). K-Means, which never saw the labels, therefore found one pure group of frauds, but it cannot separate the rest.

Hierarchical clustering (average linkage, two clusters):

Hierarchical cluster	Genuine	Fraud
0	4,998	473
1	2	0

Average-linkage clustering put 5,471 of the 5,473 transactions into one cluster and only 2 genuine outliers into the other, so with K = 2 it did not separate fraud at all.

Dendrograms.

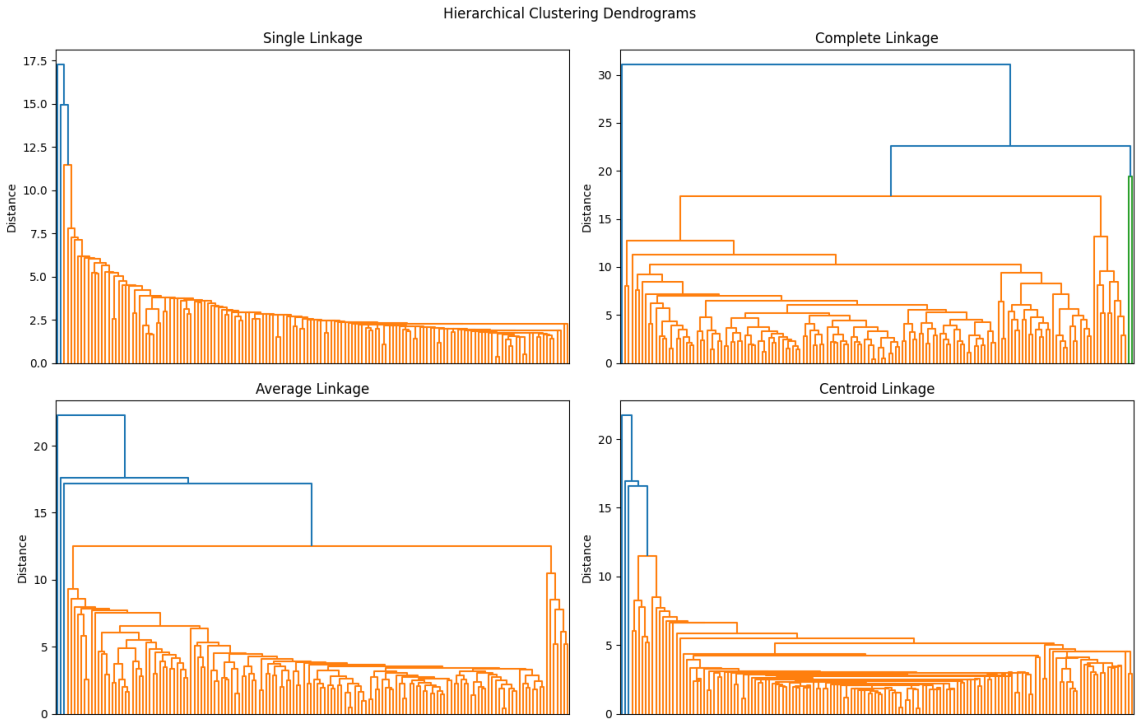


Figure 18: Dendrograms of the four linkages for 150 transactions (100 genuine and 50 fraud)

Figure 18: the **single** linkage shows the typical chaining pattern, a staircase in which points join one by one at small distances (mostly about 2 to 5) and a few outliers join late (about 11 to 17). The **complete** linkage has the largest distances; its last merge is at about 31. **Average** and **centroid** linkage look alike, with a few outlying transactions joining last at distances of about 17 to 22. In all four, a few extreme transactions are far from everything else.

11.10 Discussion

Why do the classifiers perform so well? Many V features are strongly related to Class (correlations up to about −0.79, Figure 2). Because of this, even the linear Logistic Regression reaches an F1-score of 0.8814, and the non-linear SVM improves on it by only 0.0023.

Why is recall limited to about 0.80? All five models miss 17 to 19 of 95 frauds. K-Means shows a similar picture: 282 of the 473 frauds fall into the cluster that is mostly genuine. This suggests that a part of the fraud cases looks like genuine transactions in the available features, so no model can find them without more information or a different decision threshold.

Why does Naive Bayes perform worst? It produced 22 false alarms against 1 to 4 for the other models. Gaussian Naive Bayes assumes that each feature is normally distributed and independent within each class, whereas the V features have very heavy tails (Section 5.3) and, in the rebalanced sample, are related to each other through Class. This is a plausible explanation and was not tested.

Why is the log loss of KNN so high? With k = 5, KNN can only output probabilities of 0, 0.2, 0.4, 0.6, 0.8 or 1. A confident wrong vote is punished heavily by log loss, even though accuracy and F1 are good. This is an interpretation.

Which model would be chosen? By F1, accuracy and AUC the SVM is first, but it is practically tied with Logistic Regression, which is faster, easier to explain and has the lowest log loss. The final choice in practice would depend on the costs of a missed fraud and of a false alarm.

Supervised against unsupervised. The classifiers use the labels and find about 80% of the fraud with very few false alarms. K-Means and hierarchical clustering do not use the labels; K-Means found a pure fraud cluster with 40% of the frauds, and hierarchical clustering found none. Clustering is therefore useful for exploring the structure of the data, but it is not a substitute for a classifier here.

11.11 Limitations of the Results

- The results come from a single train-test split; no cross-validation or repeated runs were done, so small differences between models are not statistically tested.
- The test set contains only 95 fraud cases, so one more or less transaction changes the recall by about one percentage point.
- The data were under-sampled to 8.64% fraud (Section 11.7); precision, accuracy and log loss on real traffic would be different.
- No hyper-parameter tuning was done (k = 5, tree depth 5 and SVM C = 1.0 were fixed in advance), so each model may be below its best performance.

- The decision threshold of 0.5 was used for all models; no threshold tuning was done.
- The features are anonymised, so the results cannot be explained in business terms.
- The regression models are strongly affected by a few very large amounts, and they can predict negative amounts.
- Confusion matrices for the models other than the SVM were not printed by the program; they were derived in Section 11.6.
- The model should not be used to make decisions about individual cardholders.

12. Conclusion

This project built and compared five classical classifiers (Logistic Regression, KNN, Gaussian Naive Bayes, Decision Tree and SVM) to detect fraud in the Kaggle Credit Card Fraud Detection dataset. The data were cleaned, explored, reduced to a sample of 473 fraud and 5,000 genuine transactions, split 80:20, standardised and evaluated with accuracy, precision, recall, F1-score, ROC AUC, log loss and a confusion matrix. The same data were also used for simple, polynomial and multiple linear regression of Amount, and for K-Means and hierarchical clustering.

12.1 Key Findings

1. **SVM was the best classifier overall**, with accuracy 0.9817, precision 0.9870, F1-score 0.8837 and ROC AUC 0.9847. **Logistic Regression** was almost identical (accuracy 0.9808, F1 0.8814, AUC 0.9826) and had the lowest log loss (0.0671).
2. Decision Tree (accuracy 0.9808) and KNN (0.9799) were close behind. **Gaussian Naive Bayes** was the weakest (accuracy 0.9644, F1 0.8000, log loss 0.8224) because of 22 false alarms.
3. All five models beat the 91.3% majority-class baseline of the test set, but **no model found more than 78 of the 95 frauds** (recall at most 0.8211).
4. The SVM confusion matrix shows that 999 of 1,000 genuine and 76 of 95 fraud transactions were classified correctly; 19 of its 20 errors are missed frauds.

5. In regression, **multiple linear regression** predicted Amount far better (R^2 = 0.8980, RMSE = 94.57) than simple (R^2 = 0.2456) or polynomial (R^2 = 0.3383) regression on V2 alone.
6. In clustering, the elbow method pointed to K = 2, and K-Means produced one cluster that contained only fraud (191 of 473 frauds). Average-linkage hierarchical clustering put almost all points into one cluster. The comparison of initialisations showed that a single K-Means run can end in a worse local minimum.
7. The EDA showed extreme class imbalance (0.17% fraud) and that features such as V14, V12 and V10 are strongly negatively correlated with fraud.

12.2 Learning Outcomes

Through this project the author practised the full workflow of predictive analytics on real data: reading the dataset documentation, cleaning, sampling, exploring, scaling, splitting, training and comparing several algorithms, interpreting a confusion matrix, evaluating regression models with MAE, MSE, RMSE and R^2, and using K-Means and hierarchical clustering. Key lessons are that accuracy alone is misleading on imbalanced data, that the choice of measure changes which model looks best, and that results on a rebalanced sample must not be mistaken for results on real traffic.

12.3 Final Remarks

The results should be read as a baseline comparison on one dataset, one data split and a rebalanced sample. They do not prove that the best-ranked model would remain best on other data or on the full, unsampled data. The conclusions are limited to what was measured.

13. Future Scope

The following items are not part of the completed work of this report. They are suggestions for extending the project.

13.1 Improvements Within the Same Framework

- **Cross-validation.** Evaluating each model on several different splits would give an average and a spread for each measure, so that the small difference between SVM and Logistic Regression can be judged properly.

- **Hyper-parameter tuning.** Systematically trying values of k for KNN, tree depth for the Decision Tree, and C and the kernel for the SVM, using validation data.
- **Threshold tuning and cost.** Choosing the decision threshold from the cost of a missed fraud against the cost of a false alarm, instead of always using 0.5.
- **Testing on the full data.** Training on the sample but testing on the original class mix, to measure precision as it would be in practice.
- **Other ways to handle imbalance.** Class weights or SMOTE [16] instead of simple under-sampling.
- **Confusion matrices and per-class reports** for all five models.

13.2 Extensions Beyond the Course Units

Methods from later courses, such as random forests, gradient boosting and neural networks, could be compared with the five baseline models. They are mentioned only as possible future work and were not implemented in this project.

13.3 Data Extensions

- Using more recent transactions or other fraud datasets to test whether the patterns hold elsewhere.
- Splitting by time (train on the earlier day, test on the later day), which is closer to real use than a random split.
- Adding richer, interpretable features such as merchant type or location, if available.

13.4 Practical Extension

The best model could be wrapped in a simple web form or notebook where a user enters a transaction and sees the fraud probability together with a warning about its limited accuracy. A real system would also need monitoring, because fraud patterns change over time [9].

Appendix B: Program Source Code

Complete Python source code supplied for this project.

```
import os
os.environ["LOKY_MAX_CPU_COUNT"] = "4"
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler,
PolynomialFeatures

from sklearn.linear_model import LogisticRegression,
LinearRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.svm import SVC

from sklearn.cluster import KMeans, AgglomerativeClustering
from scipy.cluster.hierarchy import linkage, dendrogram

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    roc_auc_score,
    log_loss,
    confusion_matrix,
    classification_report,
    roc_curve,
    mean_absolute_error,
    mean_squared_error,
    r2_score
)

print("\nLoading dataset...")
```

```
df =
pd.read_csv(r"C:\Users\DELL\Desktop\int234PredictiveProject\
creditcard.csv")
print("Dataset loaded successfully!")
print("Total records:", len(df))
print("Total columns:", len(df.columns))
print("\nFirst 5 rows:")
print(df.head())
print("\nDataset information:")
print(df.describe().T[["mean", "std", "min",
"max"]].head(10))

df = df.drop_duplicates()
df = df.dropna()
df["Class"] = df["Class"].astype(int)

print("\nMissing values:", int(df.isnull().sum().sum()))
print("Dataset after cleaning:", df.shape)
print("\nTransaction status:")
print("Genuine transactions:", (df["Class"] == 0).sum())
print("Fraud transactions:", (df["Class"] == 1).sum())

os.makedirs("project_outputs", exist_ok=True)
fraud_df = df[df["Class"] == 1]
genuine_df = df[df["Class"] == 0].sample(5000,
random_state=42)
data = pd.concat([fraud_df, genuine_df]).sample(
    frac=1, random_state=42
).reset_index(drop=True)

print("\nSampled dataset used for the project:", data.shape)
print("Genuine in sample:", (data["Class"] == 0).sum())
print("Fraud in sample:", (data["Class"] == 1).sum())

v_features = ["V" + str(i) for i in range(1, 29)]
features = ["Time", "Amount"] + v_features
X = data[features]
y = data["Class"]
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print("\nTraining records:", len(X_train))
print("Testing records:", len(X_test))

models = {
    "Logistic Regression":
LogisticRegression(max_iter=1000),
    "KNN": KNeighborsClassifier(n_neighbors=5),
    "Naive Bayes": GaussianNB(),
    "Decision Tree": DecisionTreeClassifier(max_depth=5,
random_state=42),
    "SVM": SVC(kernel="rbf", probability=True,
random_state=42)
}

results = []
trained_models = {}
model_probabilities = {}
model_predictions = {}
print("\nTraining all five models...")

for name, model in models.items():
    print("\nTraining:", name)
    model.fit(X_train_scaled, y_train)
    predictions = model.predict(X_test_scaled)
    probabilities = model.predict_proba(X_test_scaled)[:, 1]

    accuracy = accuracy_score(y_test, predictions)
    precision = precision_score(y_test, predictions,
zero_division=0)
    recall = recall_score(y_test, predictions,
zero_division=0)
```

```
    f1 = f1_score(y_test, predictions, zero_division=0)
    auc = roc_auc_score(y_test, probabilities)
    loss = log_loss(y_test, probabilities)

    results.append({
        "Model": name, "Accuracy": accuracy, "Precision":
precision,
        "Recall": recall, "F1 Score": f1, "ROC AUC": auc,
"Log Loss": loss
    })
    trained_models[name] = model
    model_probabilities[name] = probabilities
    model_predictions[name] = predictions

    print("Accuracy:", round(accuracy, 4))
    print("Precision:", round(precision, 4))
    print("Recall:", round(recall, 4))
    print("F1 Score:", round(f1, 4))
    print("ROC AUC:", round(auc, 4))
    print("Log Loss:", round(loss, 4))

results_df = pd.DataFrame(results).sort_values(by="F1
Score", ascending=False)
print("\n========== MODEL COMPARISON ==========")
print(results_df.round(4).to_string(index=False))
results_df.to_csv("project_outputs/model_comparison.csv",
index=False)

best_name = results_df.iloc[0]["Model"]
best_model = trained_models[best_name]
best_predictions = model_predictions[best_name]
print("\nBest model by F1 Score:", best_name)
print("\nClassification report:")
print(classification_report(
    y_test, best_predictions, labels=[0, 1],
    target_names=["Genuine", "Fraud"], zero_division=0
))

plt.figure(figsize=(6, 5))
```

```
plt.bar(["Genuine", "Fraud"],
data["Class"].value_counts().reindex([0, 1]).values)
plt.title("Genuine vs Fraud Transactions (Sample)")
plt.xlabel("Transaction Type")
plt.ylabel("Number of Transactions")
plt.tight_layout()
plt.savefig("project_outputs/01_class_distribution.png")
plt.show()

plt.figure(figsize=(9, 5))
plt.bar(results_df["Model"], results_df["Accuracy"])
plt.title("Accuracy Comparison of Five ML Models")
plt.xlabel("Model")
plt.ylabel("Accuracy")
plt.ylim(0, 1)
plt.xticks(rotation=25, ha="right")
plt.tight_layout()
plt.savefig("project_outputs/02_model_accuracy.png")
plt.show()

plt.figure(figsize=(9, 5))
plt.bar(results_df["Model"], results_df["F1 Score"])
plt.title("F1 Score Comparison of Five ML Models")
plt.xlabel("Model")
plt.ylabel("F1 Score")
plt.ylim(0, 1)
plt.xticks(rotation=25, ha="right")
plt.tight_layout()
plt.savefig("project_outputs/03_model_f1_score.png")
plt.show()

plt.figure(figsize=(9, 5))
plt.bar(results_df["Model"], results_df["Log Loss"])
plt.title("Log Loss Comparison of Five ML Models")
plt.xlabel("Model")
plt.ylabel("Log Loss")
plt.xticks(rotation=25, ha="right")
plt.tight_layout()
plt.savefig("project_outputs/04_model_log_loss.png")
```

```
plt.show()

plt.figure(figsize=(9, 6))
for name, probabilities in model_probabilities.items():
    fpr, tpr, thresholds = roc_curve(y_test, probabilities)
    auc = roc_auc_score(y_test, probabilities)
    plt.plot(fpr, tpr, label=f"{name} (AUC={auc:.3f})")
plt.plot([0, 1], [0, 1], linestyle="--")
plt.title("ROC Curves - Five Models")
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.legend(fontsize=8)
plt.tight_layout()
plt.savefig("project_outputs/05_roc_curves.png")
plt.show()

cm = confusion_matrix(y_test, best_predictions, labels=[0,
1])
plt.figure(figsize=(6, 5))
plt.imshow(cm, interpolation="nearest")
plt.title("Confusion Matrix - " + best_name)
plt.colorbar()
plt.xticks([0, 1], ["Genuine", "Fraud"])
plt.yticks([0, 1], ["Genuine", "Fraud"])
plt.xlabel("Predicted Status")
plt.ylabel("Actual Status")
for i in range(2):
    for j in range(2):
        plt.text(j, i, str(cm[i, j]), ha="center",
va="center")
plt.tight_layout()
plt.savefig("project_outputs/06_confusion_matrix.png")
plt.show()

correlation = data[features +
["Class"]].corr()["Class"].drop("Class")
top_corr = correlation.reindex(
```

```
correlation.abs().sort_values(ascending=False).head(12).inde
x
)
plt.figure(figsize=(9, 6))
plt.barh(top_corr.index[::-1], top_corr.values[::-1])
plt.title("Top 12 Features Correlated with Fraud")
plt.xlabel("Correlation with Class")
plt.tight_layout()
plt.savefig("project_outputs/07_correlation_with_fraud.png")
plt.show()

plt.figure(figsize=(7, 5))
plt.boxplot(
    [data.loc[data["Class"] == 0, "Amount"],
     data.loc[data["Class"] == 1, "Amount"]],
    showfliers=False
)
plt.xticks([1, 2], ["Genuine", "Fraud"])
plt.title("Transaction Amount by Type")
plt.ylabel("Amount")
plt.tight_layout()
plt.savefig("project_outputs/08_amount_distribution.png")
plt.show()

print("\n========== REGRESSION (PREDICTING TRANSACTION
AMOUNT) ==========")
reg_features = ["Time"] + v_features
Xr = data[reg_features]
yr = data["Amount"]
Xr_train, Xr_test, yr_train, yr_test = train_test_split(
    Xr, yr, test_size=0.20, random_state=42
)
amount_corr = data[reg_features +
["Amount"]].corr()["Amount"].drop("Amount")
top_feature = amount_corr.abs().idxmax()
print("Feature most correlated with Amount:", top_feature)
print("Correlation:", round(amount_corr[top_feature], 4))
```

```
regression_results = []
def evaluate_regression(name, actual, predicted):
    mae = mean_absolute_error(actual, predicted)
    mse = mean_squared_error(actual, predicted)
    rmse = np.sqrt(mse)
    r2 = r2_score(actual, predicted)
    regression_results.append({
        "Model": name, "MAE": mae, "MSE": mse, "RMSE": rmse,
"R2 Score": r2
    })

x1_train = Xr_train[[top_feature]].values
x1_test = Xr_test[[top_feature]].values
simple_model = LinearRegression()
simple_model.fit(x1_train, yr_train)
simple_pred = simple_model.predict(x1_test)
evaluate_regression("Simple Linear", yr_test, simple_pred)

poly = PolynomialFeatures(degree=2)
x1_train_poly = poly.fit_transform(x1_train)
x1_test_poly = poly.transform(x1_test)
poly_model = LinearRegression()
poly_model.fit(x1_train_poly, yr_train)
poly_pred = poly_model.predict(x1_test_poly)
evaluate_regression("Polynomial (degree 2)", yr_test,
poly_pred)

multi_model = LinearRegression()
multi_model.fit(Xr_train, yr_train)
multi_pred = multi_model.predict(Xr_test)
evaluate_regression("Multiple Linear", yr_test, multi_pred)

regression_df = pd.DataFrame(regression_results)
print(regression_df.round(4).to_string(index=False))
regression_df.to_csv("project_outputs/regression_comparison.
csv", index=False)

order = np.argsort(x1_test[:, 0])
plt.figure(figsize=(8, 5))
```

```
plt.scatter(x1_test[:, 0], yr_test, alpha=0.4,
label="Actual")
plt.plot(x1_test[order, 0], simple_pred[order], color="red",
label="Simple Linear")
plt.title("Simple Linear Regression: Amount vs " +
top_feature)
plt.xlabel(top_feature)
plt.ylabel("Amount")
plt.legend()
plt.tight_layout()
plt.savefig("project_outputs/09_simple_linear_regression.png
")
plt.show()

plt.figure(figsize=(8, 5))
plt.scatter(x1_test[:, 0], yr_test, alpha=0.4,
label="Actual")
plt.plot(x1_test[order, 0], poly_pred[order], color="green",
label="Polynomial (degree 2)")
plt.title("Polynomial Regression: Amount vs " + top_feature)
plt.xlabel(top_feature)
plt.ylabel("Amount")
plt.legend()
plt.tight_layout()
plt.savefig("project_outputs/10_polynomial_regression.png")
plt.show()

plt.figure(figsize=(7, 6))
plt.scatter(yr_test, multi_pred, alpha=0.4)
limit = max(yr_test.max(), multi_pred.max())
plt.plot([0, limit], [0, limit], linestyle="--",
color="red")
plt.title("Multiple Linear Regression: Actual vs Predicted
Amount")
plt.xlabel("Actual Amount")
plt.ylabel("Predicted Amount")
plt.tight_layout()
plt.savefig("project_outputs/11_multiple_regression.png")
plt.show()
```

```
print("\n========== CLUSTERING (UNSUPERVISED LEARNING)
==========")
cluster_scaler = StandardScaler()
cluster_data =
cluster_scaler.fit_transform(data[v_features])
inertia = []
k_values = range(1, 11)
for k in k_values:
    km = KMeans(n_clusters=k, init="k-means++", n_init=10,
random_state=42)
    km.fit(cluster_data)
    inertia.append(km.inertia_)

plt.figure(figsize=(8, 5))
plt.plot(list(k_values), inertia, marker="o")
plt.title("K-Means Elbow Method")
plt.xlabel("Number of Clusters (K)")
plt.ylabel("WCSS (Inertia)")
plt.tight_layout()
plt.savefig("project_outputs/12_kmeans_elbow.png")
plt.show()

print("\nRandom initialization vs k-means++ (single run, 3
seeds):")
for seed in [1, 2, 3]:
    km_random = KMeans(n_clusters=3, init="random",
n_init=1, random_state=seed).fit(cluster_data)
    km_plus = KMeans(n_clusters=3, init="k-means++",
n_init=1, random_state=seed).fit(cluster_data)
    print("Seed", seed, "| random inertia:",
round(km_random.inertia_, 1),
          "| k-means++ inertia:", round(km_plus.inertia_,
1))

kmeans = KMeans(n_clusters=2, init="k-means++", n_init=10,
random_state=42)
data["kmeans_cluster"] = kmeans.fit_predict(cluster_data)
print("\nK-Means clusters vs actual Class:")
```

```
print(pd.crosstab(data["kmeans_cluster"], data["Class"]))

plt.figure(figsize=(8, 6))
for cluster in sorted(data["kmeans_cluster"].unique()):
    subset = data[data["kmeans_cluster"] == cluster]
    plt.scatter(subset["V1"], subset["V2"], alpha=0.4,
label="Cluster " + str(cluster))
plt.title("K-Means Clusters (K=2) on V1 and V2")
plt.xlabel("V1")
plt.ylabel("V2")
plt.legend()
plt.tight_layout()
plt.savefig("project_outputs/13_kmeans_clusters.png")
plt.show()

agglo = AgglomerativeClustering(n_clusters=2,
linkage="average")
data["hierarchical_cluster"] =
agglo.fit_predict(cluster_data)
print("\nHierarchical (average linkage) clusters vs actual
Class:")
print(pd.crosstab(data["hierarchical_cluster"],
data["Class"]))

dendro_sample = pd.concat([
    data[data["Class"] == 0].sample(100, random_state=1),
    data[data["Class"] == 1].sample(50, random_state=1)
])
dendro_data =
StandardScaler().fit_transform(dendro_sample[v_features])
fig, axes = plt.subplots(2, 2, figsize=(14, 9))
for ax, method in zip(axes.flatten(), ["single", "complete",
"average", "centroid"]):
    Z = linkage(dendro_data, method=method)
    dendrogram(Z, ax=ax, no_labels=True)
    ax.set_title(method.capitalize() + " Linkage")
    ax.set_ylabel("Distance")
plt.suptitle("Hierarchical Clustering Dendrograms")
plt.tight_layout()
```

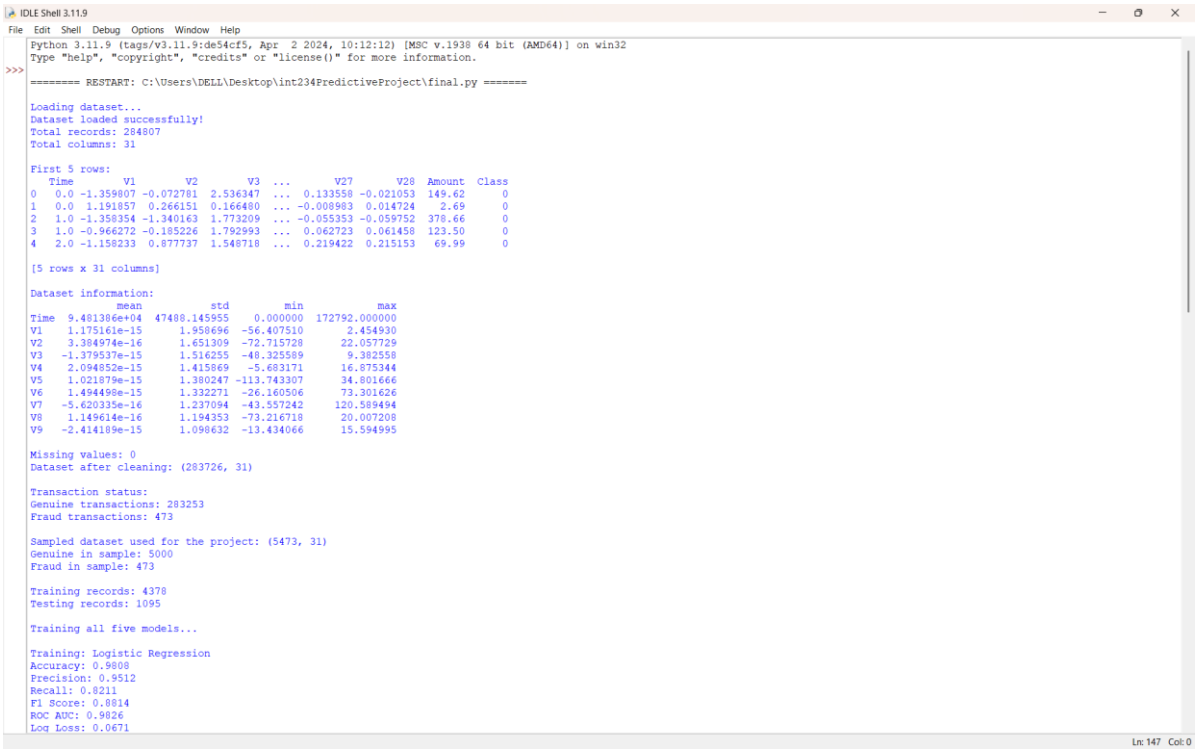
```
plt.savefig("project_outputs/14_dendrograms.png")
plt.show()

prediction_output = X_test.copy()
prediction_output["actual_status"] = y_test
prediction_output["predicted_status"] = best_predictions
prediction_output["fraud_probability"] =
model_probabilities[best_name]
prediction_output.to_csv("project_outputs/test_predictions.c
sv", index=False)

print("\n========== SAMPLE TRANSACTION PREDICTIONS
==========")
sample = X_test.sample(5, random_state=1)
sample_scaled = scaler.transform(sample)
sample_probabilities =
best_model.predict_proba(sample_scaled)[:, 1]
sample_results = best_model.predict(sample_scaled)
for i in range(len(sample)):
    print(
        "Transaction", i + 1,
        "| Amount:", round(sample["Amount"].iloc[i], 2),
        "| Fraud probability:",
round(sample_probabilities[i] * 100, 2), "%",
        "| Prediction:", "FRAUD" if sample_results[i] == 1
else "GENUINE",
        "| Actual:", "FRAUD" if y_test.loc[sample.index[i]]
== 1 else "GENUINE"
    )

print("\nProject execution completed.")
print("All 14 graphs, model comparison, regression results
and predictions are saved in the project_outputs folder.")
```

Appendix C: Program Execution Screenshots



```
IDLE Shell 3.11.9
File  Edit  Shell  Debug  Options  Window  Help
Python 3.11.9 (tags/v3.11.9:de54cf5, Apr  2 2024, 10:12:12) [MSC v.1938 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
======== RESTART: C:\Users\DELL\Desktop\int234PredictiveProject\final.py =======

Loading dataset...
Dataset loaded successfully!
Total records: 284807
Total columns: 31

First 5 rows:
   Time        V1        V2        V3  ...       V27       V28  Amount  Class
0   0.0 -1.359807 -0.072781  2.536347  ...  0.133558 -0.021053  149.62      0
1   0.0  1.191857  0.266151  0.166480  ... -0.008983  0.014724    2.69      0
2   1.0 -1.358354 -1.340163  1.773209  ... -0.055353 -0.059752  378.66      0
3   1.0 -0.966272 -0.185226  1.792993  ...  0.062723  0.061458  123.50      0
4   2.0 -1.158233  0.877737  1.548718  ...  0.219422  0.215153   69.99      0

[5 rows x 31 columns]

Dataset information:
                mean           std        min            max
Time    9.481386e+04  47488.145955    0.000000  172792.000000
V1      1.175161e-15      1.958696  -56.407510       2.454930
V2      3.384974e-16      1.651309  -72.715728      22.057729
V3     -1.379537e-15      1.516255  -48.325589       9.382558
V4      2.094852e-15      1.415869   -5.683171      16.875344
V5      1.021879e-15      1.380247 -113.743307      34.801666
V6      1.494498e-15      1.332271  -26.160506      73.301626
V7     -5.620335e-16      1.237094  -43.557242     120.589494
V8      1.149614e-16      1.194353  -73.216718      20.007208
V9     -2.414189e-15      1.098632  -13.434066      15.594995

Missing values: 0
Dataset after cleaning: (283726, 31)

Transaction status:
Genuine transactions: 283253
Fraud transactions: 473

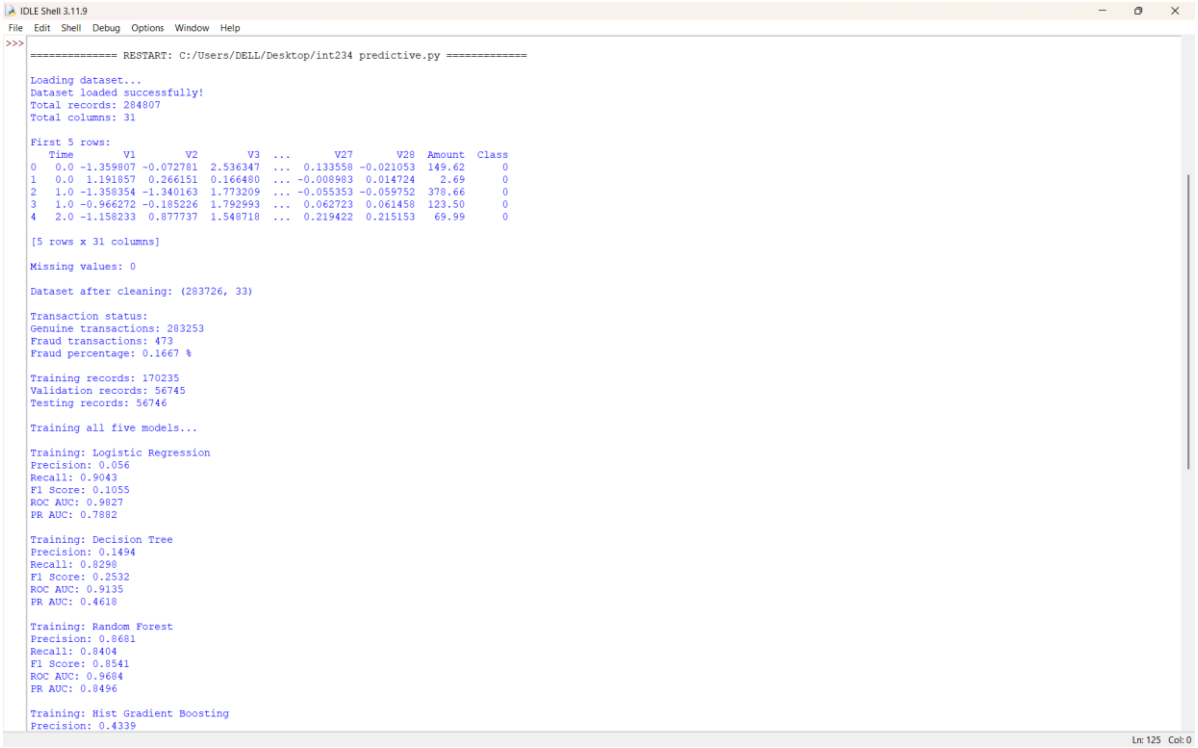
Sampled dataset used for the project: (5473, 31)
Genuine in sample: 5000
Fraud in sample: 473

Training records: 4378
Testing records: 1095

Training all five models...

Training: Logistic Regression
Accuracy: 0.9808
Precision: 0.9512
Recall: 0.8211
F1 Score: 0.8814
ROC AUC: 0.9826
Log Loss: 0.0671
                                                                    Ln: 147  Col: 0
```

Program execution: sampled dataset and initial model metrics



```
IDLE Shell 3.11.9
File  Edit  Shell  Debug  Options  Window  Help
>>>
============= RESTART: C:/Users/DELL/Desktop/int234 predictive.py =============

Loading dataset...
Dataset loaded successfully!
Total records: 284807
Total columns: 31

First 5 rows:
   Time        V1        V2        V3  ...       V27       V28  Amount  Class
0   0.0 -1.359807 -0.072781  2.536347  ...  0.133558 -0.021053  149.62      0
1   0.0  1.191857  0.266151  0.166480  ... -0.008983  0.014724    2.69      0
2   1.0 -1.358354 -1.340163  1.773209  ... -0.055353 -0.059752  378.66      0
3   1.0 -0.966272 -0.185226  1.792993  ...  0.062723  0.061458  123.50      0
4   2.0 -1.158233  0.877737  1.548718  ...  0.219422  0.215153   69.99      0

[5 rows x 31 columns]

Missing values: 0

Dataset after cleaning: (283726, 33)

Transaction status:
Genuine transactions: 283253
Fraud transactions: 473
Fraud percentage: 0.1667 %

Training records: 170235
Validation records: 56745
Testing records: 56746

Training all five models...

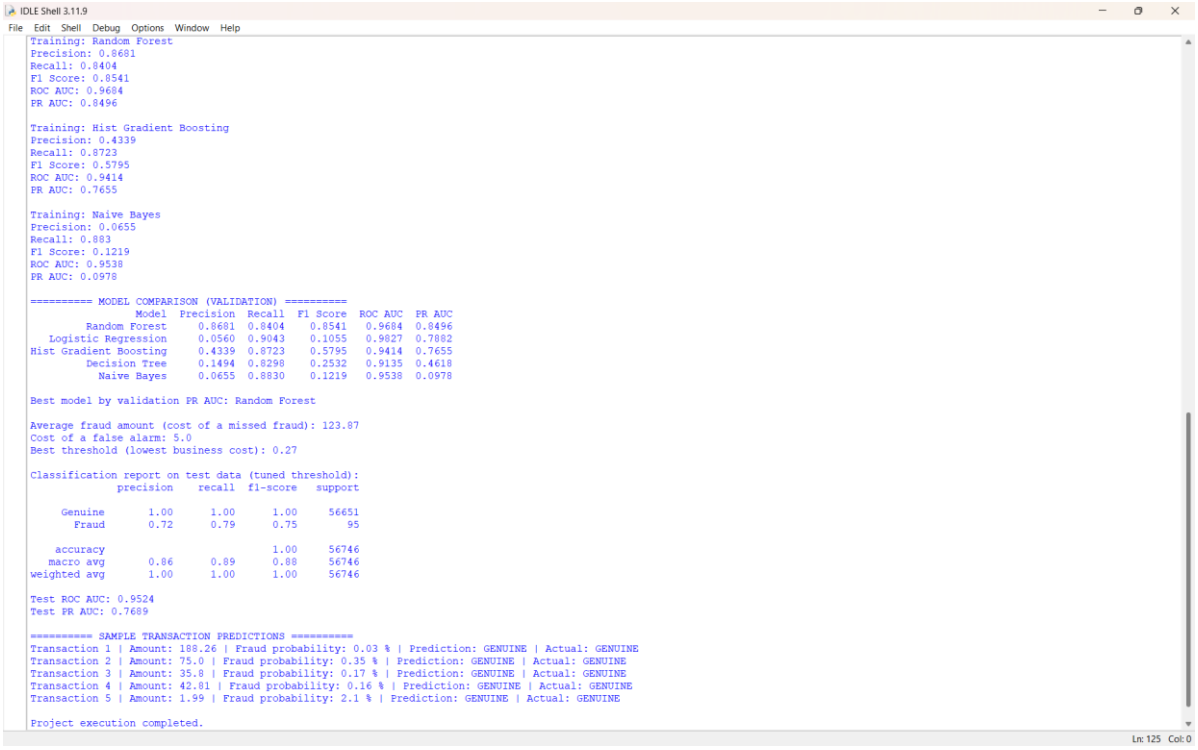
Training: Logistic Regression
Precision: 0.056
Recall: 0.9043
F1 Score: 0.1055
ROC AUC: 0.9827
PR AUC: 0.7882

Training: Decision Tree
Precision: 0.1494
Recall: 0.8298
F1 Score: 0.2532
ROC AUC: 0.9135
PR AUC: 0.4618

Training: Random Forest
Precision: 0.8681
Recall: 0.8404
F1 Score: 0.8541
ROC AUC: 0.9684
PR AUC: 0.8496

Training: Hist Gradient Boosting
Precision: 0.4339
                                                                    Ln: 125  Col: 0
```

Program execution: dataset loading, cleaning and model training


IDLE Shell 3.11.9
File Edit Shell Debug Options Window Help
Training: Random Forest
Precision: 0.8681
Recall: 0.8404
F1 Score: 0.8541
ROC AUC: 0.9684
PR AUC: 0.8496
Training: Hist Gradient Boosting
Precision: 0.4339
Recall: 0.8723
F1 Score: 0.5795
ROC AUC: 0.9414
PR AUC: 0.7655
Training: Naive Bayes
Precision: 0.0655
Recall: 0.883
F1 Score: 0.1219
ROC AUC: 0.9538
PR AUC: 0.0978
========== MODEL COMPARISON (VALIDATION) ==========
Model Precision Recall F1 Score ROC AUC PR AUC
Random Forest 0.8681 0.8404 0.8541 0.9684 0.8496
Logistic Regression 0.0560 0.9043 0.1055 0.9827 0.7882
Hist Gradient Boosting 0.4339 0.8723 0.5795 0.9414 0.7655
Decision Tree 0.1494 0.8298 0.2532 0.9135 0.4618
Naive Bayes 0.0655 0.8830 0.1219 0.9538 0.0978
Best model by validation PR AUC: Random Forest
Average fraud amount (cost of a missed fraud): 123.87
Cost of a false alarm: 5.0
Best threshold (lowest business cost): 0.27
Classification report on test data (tuned threshold):
precision recall f1-score support
Genuine 1.00 1.00 1.00 56651
Fraud 0.72 0.79 0.75 95
accuracy 1.00 56746
macro avg 0.86 0.89 0.88 56746
weighted avg 1.00 1.00 1.00 56746
Test ROC AUC: 0.9524
Test PR AUC: 0.7689
========== SAMPLE TRANSACTION PREDICTIONS ==========
Transaction 1 | Amount: 188.26 | Fraud probability: 0.03 % | Prediction: GENUINE | Actual: GENUINE
Transaction 2 | Amount: 75.0 | Fraud probability: 0.35 % | Prediction: GENUINE | Actual: GENUINE
Transaction 3 | Amount: 35.8 | Fraud probability: 0.17 % | Prediction: GENUINE | Actual: GENUINE
Transaction 4 | Amount: 42.81 | Fraud probability: 0.16 % | Prediction: GENUINE | Actual: GENUINE
Transaction 5 | Amount: 1.99 | Fraud probability: 2.1 % | Prediction: GENUINE | Actual: GENUINE
Project execution completed.
Ln: 125 Col: 0